

tractable probabilistic modeling with probabilistic circuits

antonio vergari (he/him)



@nolovedeep learning

4th Jun 2026 - **Reliable and Efficient Neuro-Symbolic AI**

thanks to...



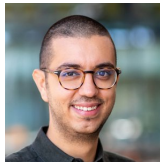
Lorenzo Loconte
U of Edinburgh



Antonio Mari
EPFL



Rickey Liang
Saarland U



Gennaro Gala
TU Eindhoven



Adrian Javaloy
U of Edinburgh

and moar...

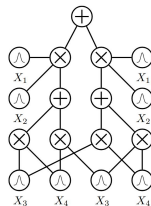
april

`april-tools.github.io`

april

***autonomous &
provably
reliable
intelligent
learners***

yesterday's topic...



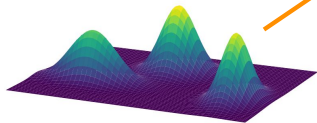
**(deep)
circuits**

$$(d \rightarrow b) \wedge (e \rightarrow b)$$

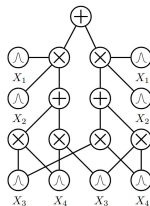
$$\vdash (\neg d \vee b) \wedge (\neg e \vee b)$$

$$\vdash b \vee (\neg d \wedge \neg e)$$

**logical
formulas
& constraints**



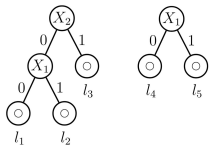
**(hierarchical)
mixtures**



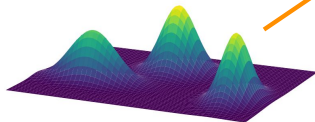
**(deep)
circuits**

$$\begin{aligned} & (d \rightarrow b) \wedge (e \rightarrow b) \\ & \vdash (\neg d \vee b) \wedge (\neg e \vee b) \\ & \vdash b \vee (\neg d \wedge \neg e) \end{aligned}$$

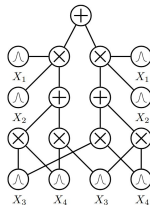
**logical
formulas
& constraints**



decision trees



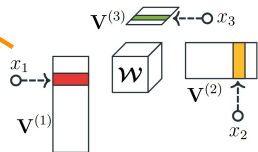
**(hierarchical)
mixtures**



**(deep)
circuits**

$$\begin{aligned} &(d \rightarrow b) \wedge (e \rightarrow b) \\ &\vdash (\neg d \vee b) \wedge (\neg e \vee b) \\ &\vdash b \vee (\neg d \wedge \neg e) \end{aligned}$$

**logical
formulas
& constraints**



**(hierarchical)
tensor factorizations**

When?



Ground Truth

e.g. predict shortest path in a map

When?



given $\mathbf{x}$ // e.g. a tile map

Ground Truth

nesy structured output prediction (SOP) tasks

When?



Ground Truth

given $\mathbf{x}$ // e.g. a tile map

find $\mathbf{y}^* = \operatorname{argmax}_{\mathbf{y}} p_{\theta}(\mathbf{y} \mid \mathbf{x})$ // e.g. a configurations of edges in a grid

nesy structured output prediction (SOP) tasks

When?



Ground Truth

given $\mathbf{x}$ // e.g. a tile map

find $\mathbf{y}^* = \operatorname{argmax}_{\mathbf{y}} p_{\theta}(\mathbf{y} \mid \mathbf{x})$ // e.g. a configurations of edges in a grid
s.t. $\mathbf{y} \models \mathbf{K}$ // e.g., that form a valid path

nesy structured output prediction (SOP) tasks

When?



Ground Truth

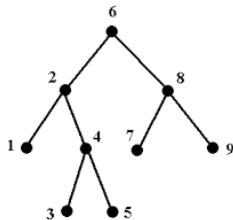
given $\mathbf{x}$ // e.g. a tile map

find $\mathbf{y}^* = \operatorname{argmax}_{\mathbf{y}} p_{\theta}(\mathbf{y} \mid \mathbf{x})$ // e.g. a configurations of edges in a grid
s.t. $\mathbf{y} \models \mathbf{K}$ // e.g., that form a valid path

// for a 12×12 grid, 2^{144} states but only 10^{10} valid ones!

nesy structured output prediction (SOP) tasks

When?



given $\mathbf{x}$ // e.g. a feature map

find $\mathbf{y}^* = \operatorname{argmax}_{\mathbf{y}} p_{\theta}(\mathbf{y} \mid \mathbf{x})$ // e.g. labels of classes

s.t. $\mathbf{y} \models \mathbf{K}$ // e.g., constraints over superclasses

$$\mathbf{K} : (Y_{\text{cat}} \implies Y_{\text{animal}}) \wedge (Y_{\text{dog}} \implies Y_{\text{animal}})$$

hierarchical multi-label classification

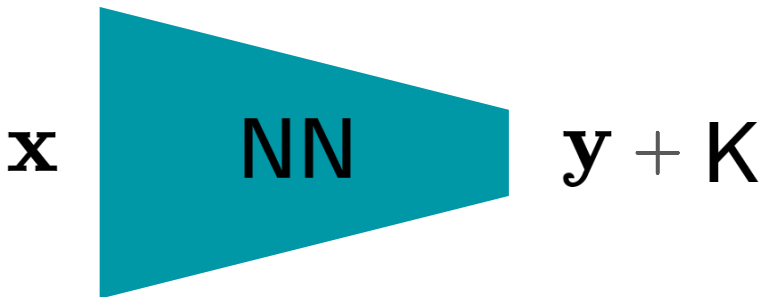
When?



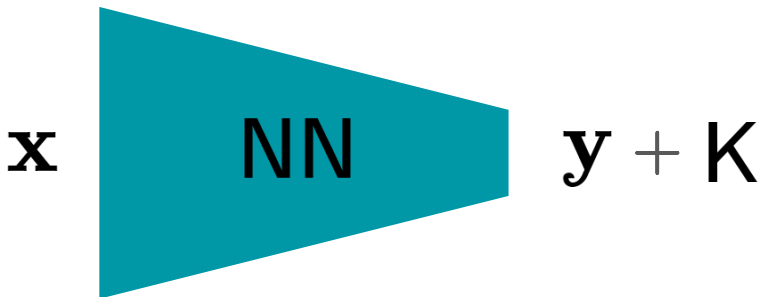
given $\mathbf{x}$ // e.g. a user preference over $K - N$ sushi types
find $\mathbf{y}^* = \operatorname{argmax}_{\mathbf{y}} p_{\theta}(\mathbf{y} \mid \mathbf{x})$ // e.g. prefs over N more types
s.t. $\mathbf{y} \models \mathbf{K}$ // e.g., output valid rankings

user preference learning

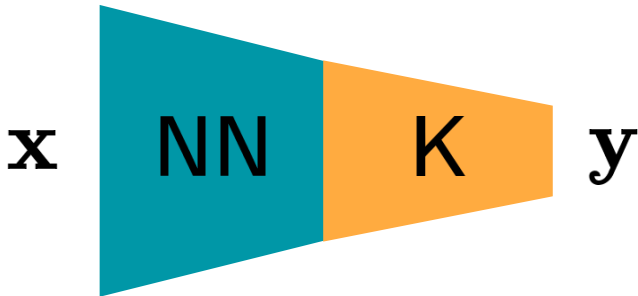
Choi, Van den Broeck, and Darwiche, "Tractable learning for structured probability spaces: A case study in learning preference distributions",
Twenty-Fourth International Joint Conference on Artificial Intelligence (IJCAI), 2015



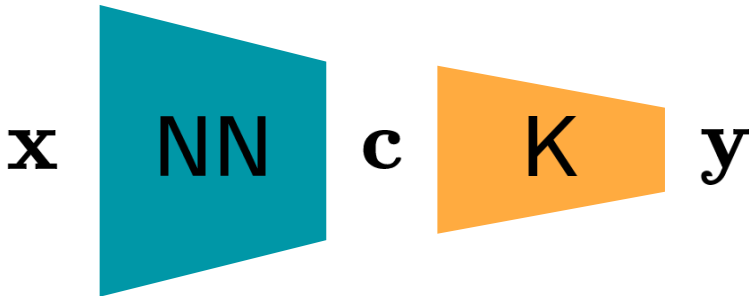
how to comply to constraint K ?



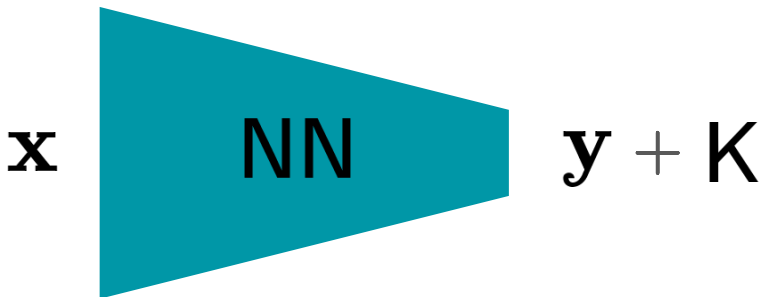
inject constraint $\mathbf{K}$ in a loss $\mathcal{L}(\mathbf{y}, \mathbf{K})$



embedding K in the neural network



NeSy concept bottleneck models



inject constraint $\mathbf{K}$ in a loss $\mathcal{L}(\mathbf{y}, \mathbf{K})$

A Semantic Loss Function for Deep Learning with Symbolic Knowledge

Jingyi Xu¹ Zilu Zhang² Tal Friedman¹ Yitao Liang¹ Guy Van den Broeck¹

the “mother” of NeSy (ICML 2018)

semantic losses

or how to inject constraints?

$$\max p_{\theta}(\mathbf{K}(\mathbf{y}))$$

maximise the probability of the constraint to hold!

semantic losses

or how to inject constraints?

$$\min -\log p_{\theta}(\mathbf{K}(\mathbf{y}))$$

minimize the semantic loss

computing the probability of logical formulas

$$p_{\theta}(\mathbf{K}(\mathbf{y})) = \mathbb{E}_{\mathbf{y} \sim q(\mathbf{y}|\mathbf{x})}[\mathbb{1}\{\mathbf{y} \models \mathbf{K}\}] = \mathbb{E}_{\mathbf{y} \sim q(\mathbf{y}|\mathbf{x})}[c(\mathbf{y})]$$

computing the probability of formula $\mathbf{K}$

WMC

computing the probability of logical formulas

$$\mathbb{E}_{\mathbf{y} \sim q(\mathbf{y}|\mathbf{x})}[\mathbb{1}\{\mathbf{y} \models \mathbf{K}\}] = \sum_{\mathbf{y}} q(\mathbf{y} \mid \mathbf{x}) \mathbb{1}\{\mathbf{y} \models \mathbf{K}\} = \sum_{\mathbf{y} \models \mathbf{K}} q(\mathbf{y} \mid \mathbf{x})$$

computing the **weighted model count** (WMC) of $\mathbf{K}$

computing the probability of logical formulas

$$\mathbb{E}_{\mathbf{y} \sim q(\mathbf{y}|\mathbf{x})}[\mathbb{1}\{\mathbf{y} \models \mathbf{K}\}] = \sum_{\mathbf{y} \models \mathbf{K}} \prod_{i: \mathbf{y} \models y_i} q(y_i \mid \mathbf{x}) \prod_{i: \mathbf{y} \models \neg y_i} (1 - q(y_i \mid \mathbf{x}))$$

assuming independence of $\mathbf{y}$ (but be careful!)

van Krieken et al., "On the Independence Assumption in Neurosymbolic Learning", ICML, 2024
Xu et al., "A Semantic Loss Function for Deep Learning with Symbolic Knowledge", ICML, 2018

WMC

computing the probability of logical formulas

$$\mathbb{E}_{\mathbf{y} \sim q(\mathbf{y}|\mathbf{x})} [\mathbb{1}\{\mathbf{y} \models \mathbf{K}\}] = \sum_{\mathbf{y} \models \mathbf{K}} \prod_{i: \mathbf{y} \models y_i} q(y_i \mid \mathbf{x}) \prod_{i: \mathbf{y} \models \neg y_i} (1 - q(y_i \mid \mathbf{x}))$$

but WMC is in general #P-hard...

van Krieken et al., "On the Independence Assumption in Neurosymbolic Learning", ICML, 2024

Xu et al., "A Semantic Loss Function for Deep Learning with Symbolic Knowledge", ICML, 2018

goal

***Can we encode K
to yield a tractable WMC?***

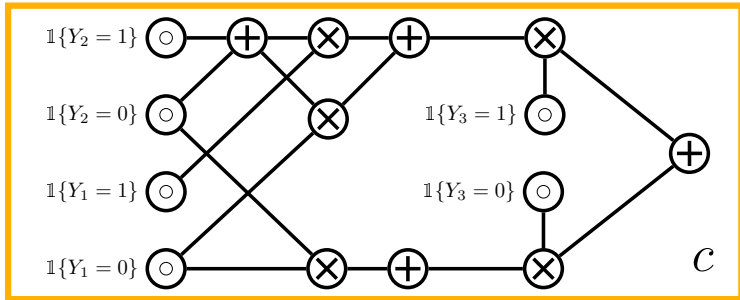
goal

***Can we encode K
to yield a tractable WMC?
yes, with **circuits!*****

goal

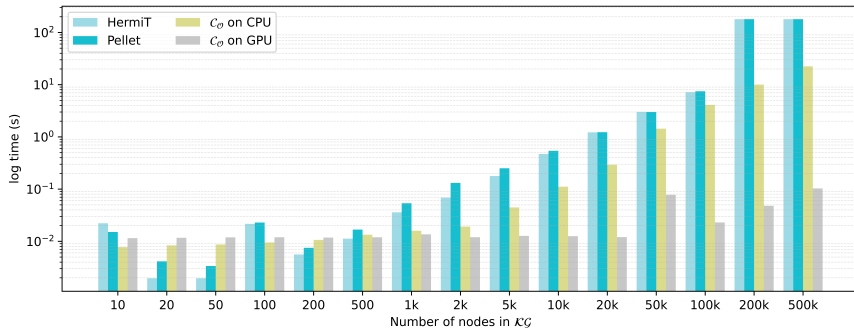
***Can we encode K
to yield a tractable WMC?
yes, with **circuits**,
if **knowledge compilation** does not explode!***

semantic loss



compiling logical formulas into circuits (as seen yesterday)

compiling ontologies



even faster than custom reasoners

Lazzari, Presutti, and Vergari, "To Neuro-Symbolic Classification and Beyond by Compiling Description Logic Ontologies to Probabilistic Circuits", 2026

LOGICALLY CONSISTENT LANGUAGE MODELS VIA NEURO-SYMBOLIC INTEGRATION

Diego Calanzone*

DISI, University of Trento

`diego.calanzone@studenti.unitn.it`

Stefano Teso



CIMeC & DISI, University of Trento

`stefano.teso@unitn.it`

Antonio Vergari



School of Informatics, University of Edinburgh

`avergari@ed.ac.uk`

MODEL	TRAIN	PPL	CONSISTENCY			SELF-CONSISTENCY			AVG
			FAC	IMP	REV	NEG	IMP	REV	
LLAMA-2-7B ZERO SHOT		62.41	0.39	0.52	0.13	0.42	0.30	0.15	0.32
LLAMA-2-7B FEW SHOT		52.30	0.53	0.71	0.34	0.38	0.48	0.47	0.48
LLAMA-2-7B CoT		52.30	0.52	0.64	0.67	0.40	0.64	0.67	0.59
LLAMA-2-70B ZERO SHOT		44.90	0.47	0.69	0.81	0.13	0.31	0.91	0.55
LLAMA-2-7B + XENT	T1+T2	116.85	0.25	0.46	0.01	0.07	0.81	0.01	0.27
LoCo-LLAMA-2-7B (NEG)	T1	62.21	0.44	0.65	0.43	0.96	0.28	0.36	0.52
LoCo-LLAMA-2-7B (F-IMP)	T1	67.15	0.99	0.99	0.07	0.00	0.99	0.07	0.51
LoCo-LLAMA-2-7B (SUPER)	T1	62.23	0.74	0.77	0.77	0.87	0.71	0.77	0.77

greatly improving (self-)consistency

SL recipe

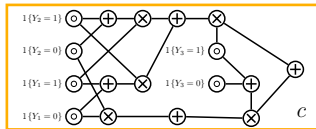
$$K : (Y_1 = 1 \implies Y_3 = 1)$$

$$\wedge (Y_2 = 1 \implies Y_3 = 1)$$

1) Take a
logical constraint

SL recipe

$$K : (Y_1 = 1 \implies Y_3 = 1) \\ \wedge (Y_2 = 1 \implies Y_3 = 1)$$

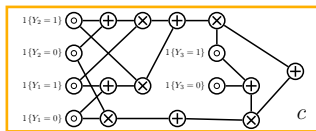


1) Take a
logical constraint

2) Compile it into
a constraint circuit

SL recipe

$$K : (Y_1 = 1 \implies Y_3 = 1) \\ \wedge (Y_2 = 1 \implies Y_3 = 1)$$



1) Take a
logical constraint

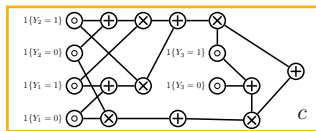
2) Compile it into
a constraint circuit

$$-\log \text{WMC}(K_i, p_\theta)$$

3) minimize the semantic loss

SL recipe

$$K : (Y_1 = 1 \implies Y_3 = 1) \\ \wedge (Y_2 = 1 \implies Y_3 = 1)$$



$$-\log \text{WMC}(K_i, p_\theta)$$

1) Take a
logical constraint

2) Compile it into
a constraint circuit

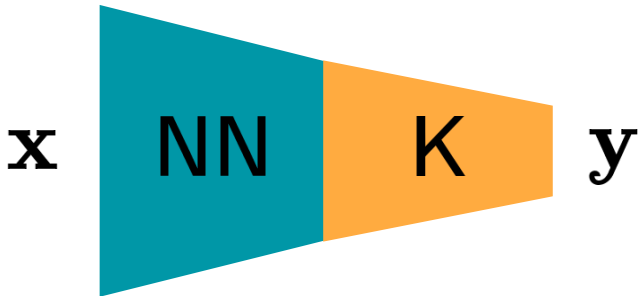
3) minimize the semantic loss

4) train end-to-end by sgd!

nice...

...but!

***no **guarantees** to satisfy
constraints at test time...***



embedding K in the neural network

Semantic Probabilistic Layers for Neuro-Symbolic Learning

Kareem Ahmed
CS Department
UCLA
ahmedk@cs.ucla.edu

Stefano Teso
CIMEC and DISI
University of Trento
stefano.teso@unitn.it

Kai-Wei Chang
CS Department
UCLA
kwchang@cs.ucla.edu

Guy Van den Broeck
CS Department
UCLA
guyvdb@cs.ucla.edu

Antonio Vergari
School of Informatics
University of Edinburgh
avergari@ed.ac.uk

a “shield” layer
at NeurIPS 2022

$$q(x)c(x)$$

products of experts

$$q(x)c(x)$$

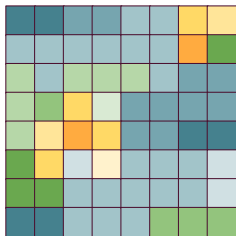
products of experts

who knows them?

$$q(x)c(x)$$

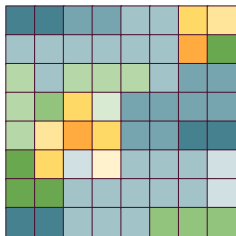
products of experts

who **loves** them?

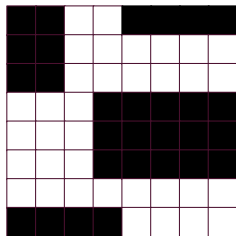


$q(\mathbf{x})$

start from a distribution $q(x_1, x_2) \dots$

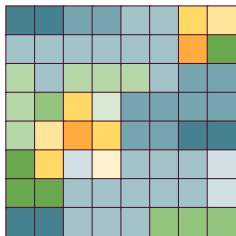


$q(\mathbf{x})$

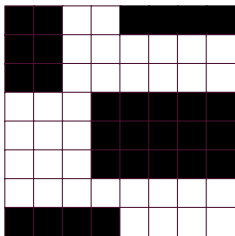


$c(\mathbf{x})$

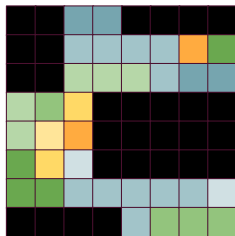
...and cut its support by a constraint $c(x_1, x_2)$



$q(\mathbf{x})$

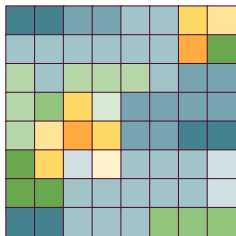


$c(\mathbf{x})$

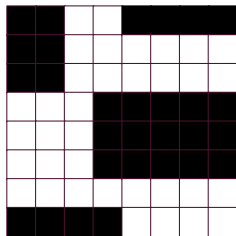


$q(\mathbf{x}) \cdot c(\mathbf{x})$

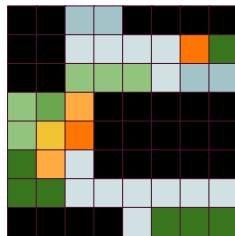
by multiplying them $q(\mathbf{x})c(\mathbf{x})...$



$q(\mathbf{x})$

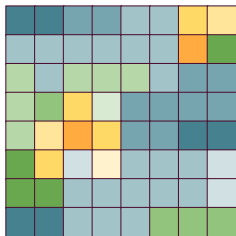


$c(\mathbf{x})$

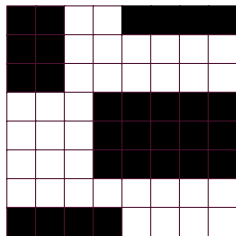


$\frac{q(\mathbf{x}) \cdot c(\mathbf{x})}{\sum_{\mathbf{x}} q(\mathbf{x}) \cdot c(\mathbf{x})}$

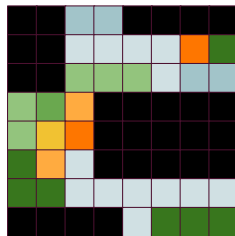
and then renormalize them



$q(\mathbf{x})$



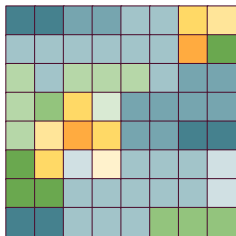
$c(\mathbf{x})$



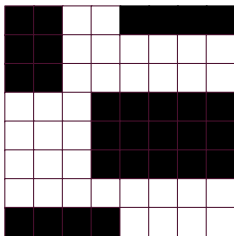
$\frac{q(\mathbf{x}) \cdot c(\mathbf{x})}{\sum_{\mathbf{x}} q(\mathbf{x}) \cdot c(\mathbf{x})}$

and then renormalize them

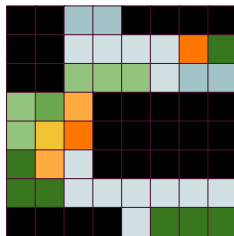
that's the tricky part...!



$q(\mathbf{x})$

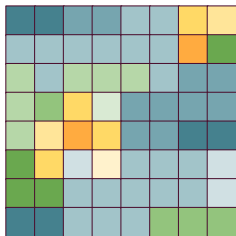


$c(\mathbf{x})$

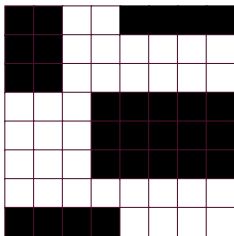


$\frac{q(\mathbf{x}) \cdot c(\mathbf{x})}{\sum_{\mathbf{x}} q(\mathbf{x}) \cdot c(\mathbf{x})}$

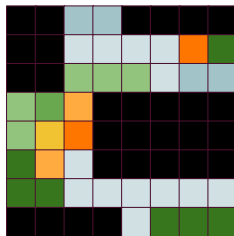
**states with zero probability will never be predicted
(*nor sampled*)**



$q(\mathbf{x})$



$c(\mathbf{x})$



$$\frac{q(\mathbf{x}) \cdot c(\mathbf{x})}{\sum_{\mathbf{x}} q(\mathbf{x}) \cdot c(\mathbf{x})}$$

but let's have a closer look at that denominator...
(have you seen it somewhere else?)

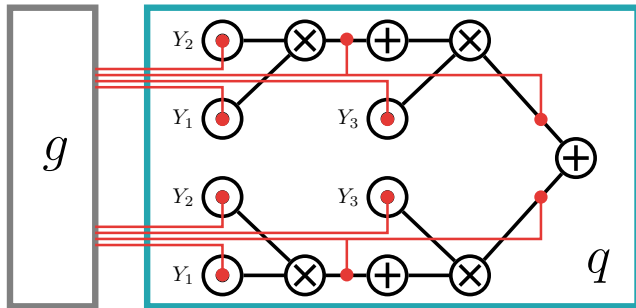
goal

*can we design q and c
to be **deep computational graphs**
yet yielding a tractable product?*

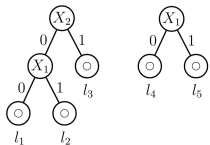
goal

*can we design q and c
to be **deep computational graphs**
yet yielding a tractable product?*

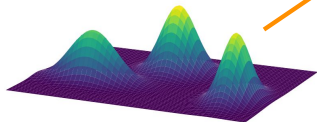
*yes! as **circuits!***



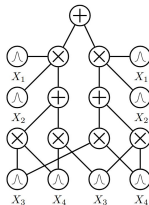
and a conditional circuit $q(y; \Theta = g(z))$



decision trees



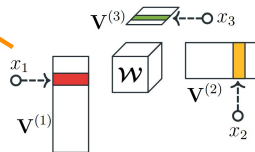
**(hierarchical)
mixtures**



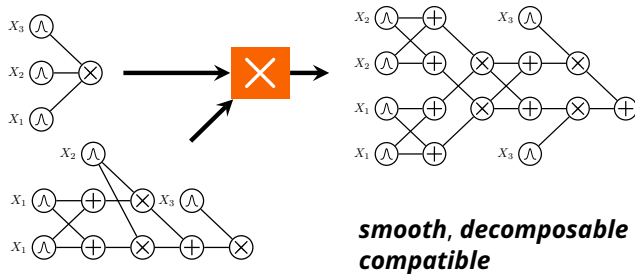
**(deep)
circuits**

$$\begin{aligned} & (d \rightarrow b) \wedge (e \rightarrow b) \\ & \vdash (\neg d \vee b) \wedge (\neg e \vee b) \\ & \vdash b \vee (\neg d \wedge \neg e) \end{aligned}$$

**logical
formulas
& constraints**



**(hierarchical)
tensor factorizations**



how to efficiently multiply (and renormalize) two PCs?



learning & reasoning with circuits in pytorch

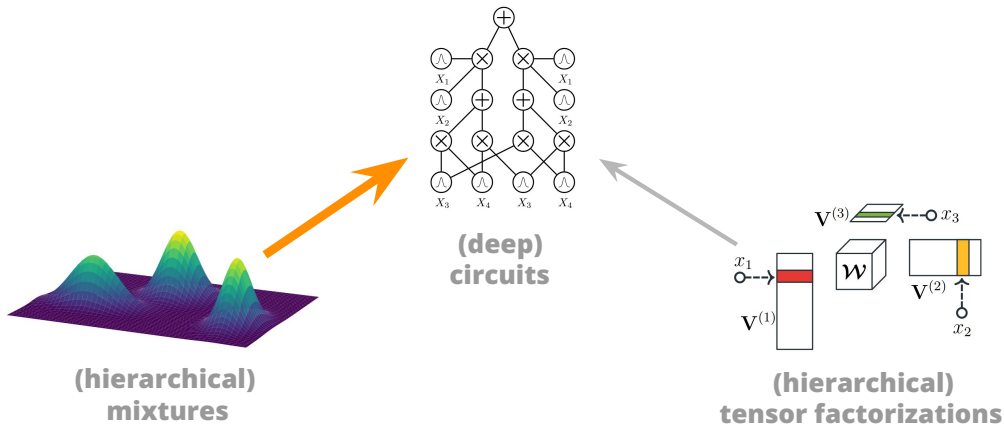
`github.com/april-tools/circuit`

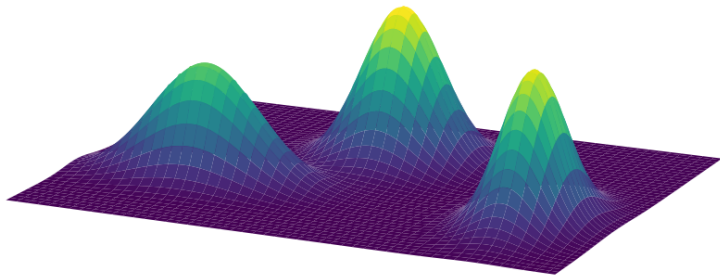
compositional inference /



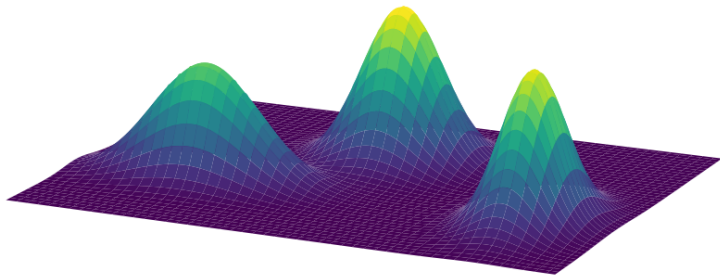
```
1 from cirq.symbolic.functional import integrate, multiply
2
3 # two deep circuits
4 q = build_symbolic_circuit(...)
5 c = build_symbolic_circuit(...)
6
7 #
8 # compute the partition function of  $q * c$ 
9 def renormalize(q, c):
10     p = multiply(q, c)
11     return integrate(p)
```

compile mixtures into circuits...





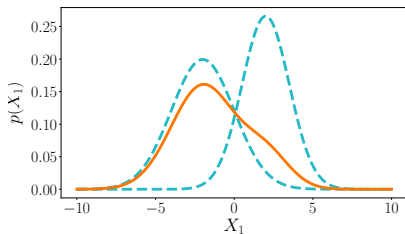
who knows mixture models?



*who **loves** mixture models?*

GMMs

as computational graphs

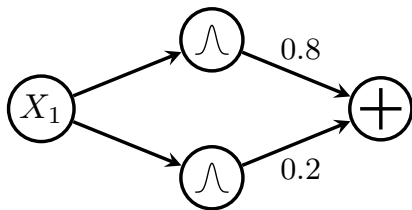


$$p(X) = w_1 \cdot p_1(X_1) + w_2 \cdot p_2(X_1)$$

⇒ *translating inference to data structures...*

GMMs

as computational graphs

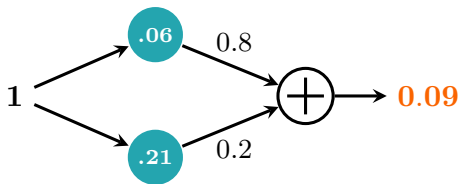


$$p(X_1) = 0.2 \cdot p_1(X_1) + 0.8 \cdot p_2(X_1)$$

$\Rightarrow$...e.g., as a weighted sum unit over Gaussian input distributions

GMMs

as computational graphs

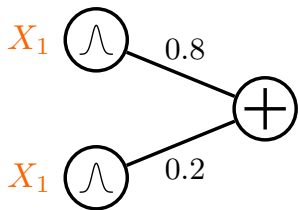


$$p(X = 1) = 0.2 \cdot p_1(X_1 = 1) + 0.8 \cdot p_2(X_1 = 1)$$

$\Rightarrow$ inference = feedforward evaluation

GMMs

as computational graphs

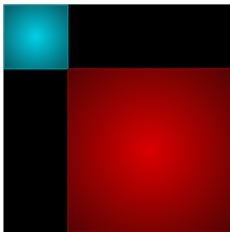


A simplified notation:

$\Rightarrow$ **scopes** attached to inputs
 $\Rightarrow$ edge directions omitted

GMMs

as computational graphs

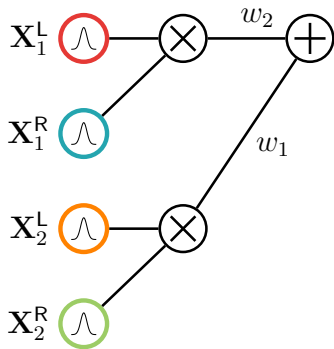


$$p(\mathbf{X}) = w_1 \cdot p_1(\mathbf{X}_1^L) \cdot p_1(\mathbf{X}_1^R) + \\ w_2 \cdot p_2(\mathbf{X}_2^L) \cdot p_2(\mathbf{X}_2^R)$$

$\Rightarrow$ local factorizations...

GMMs

as computational graphs



$$p(\mathbf{X}) = w_1 \cdot p_1(\mathbf{X}_1^L) \cdot p_1(\mathbf{X}_1^R) + w_2 \cdot p_2(\mathbf{X}_2^L) \cdot p_2(\mathbf{X}_2^R)$$

$\Rightarrow$...are product units

probabilistic circuits (PCs)

a grammar for tractable computational graphs

I. A simple tractable function is a circuit



*e.g., a multivariate Gaussian or small
neural network*

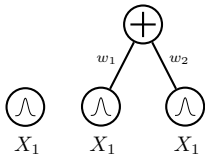


X_1

probabilistic circuits (PCs)

a grammar for tractable computational graphs

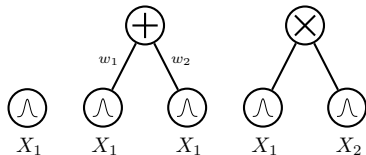
- I. *A simple tractable function is a circuit*
- II. *A weighted combination of circuits is a circuit*



probabilistic circuits (PCs)

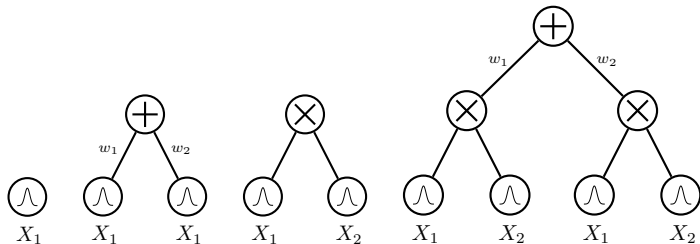
a grammar for tractable computational graphs

- I. A simple tractable function is a circuit
- II. A weighted combination of circuits is a circuit
- III. A product of circuits is a circuit



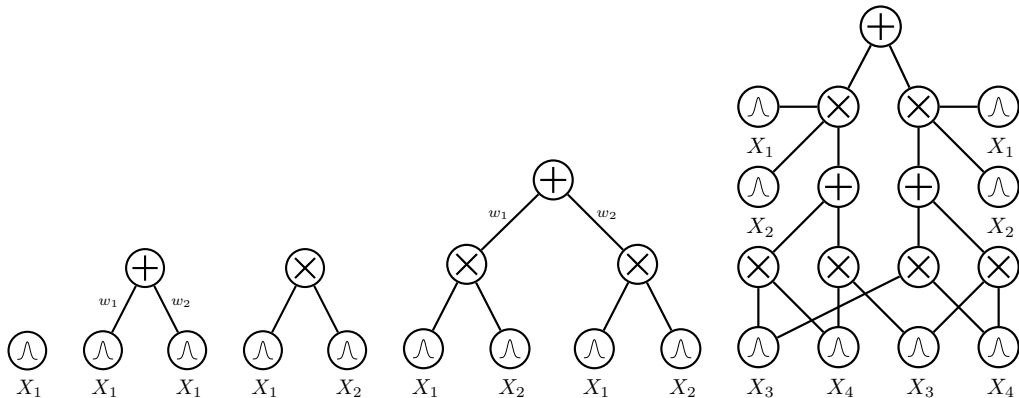
probabilistic circuits (PCs)

a grammar for tractable computational graphs



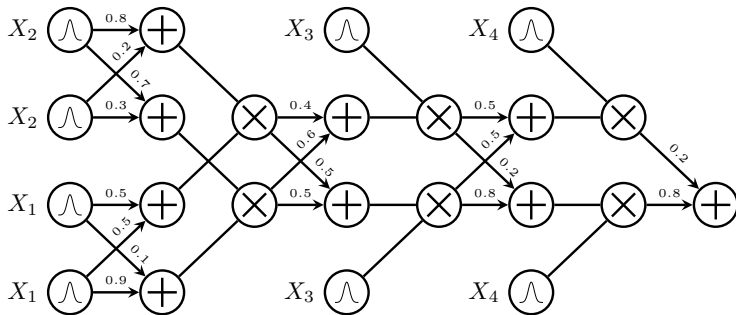
probabilistic circuits (PCs)

a grammar for tractable computational graphs



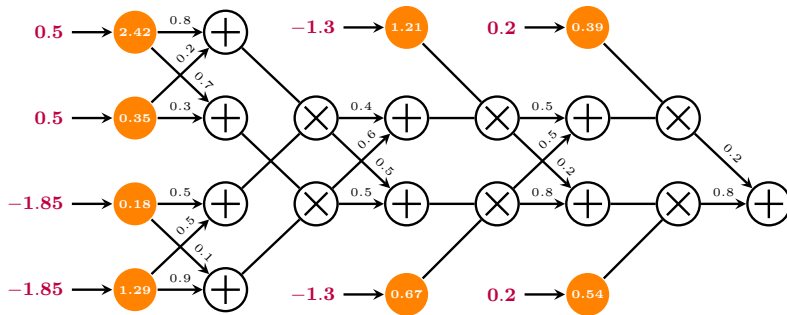
probabilistic queries = ***feedforward*** evaluation

$$p(X_1 = -1.85, X_2 = 0.5, X_3 = -1.3, X_4 = 0.2)$$



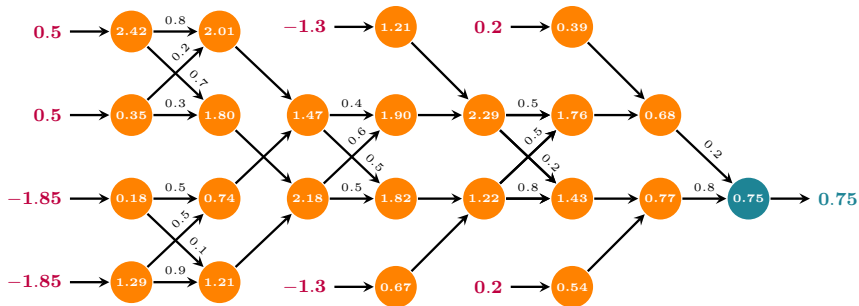
probabilistic queries = ***feedforward*** evaluation

$$p(X_1 = -1.85, X_2 = 0.5, X_3 = -1.3, X_4 = 0.2)$$



probabilistic queries = *feedforward* evaluation

$$p(X_1 = -1.85, X_2 = 0.5, X_3 = -1.3, X_4 = 0.2) = 0.75$$



probabilistic circuits (PCs)

a tensorized definition

I. *A set of tractable functions is a circuit layer*



probabilistic circuits (PCs)

a tensorized definition

- I. A set of tractable functions is a circuit layer
- II. A linear projection of a layer is a circuit layer

$$c(\mathbf{x}) = \mathbf{W}l(\mathbf{x})$$



probabilistic circuits (PCs)

a tensorized definition

- I. A set of tractable functions is a circuit layer
- II. A linear projection of a layer is a circuit layer

$$c(\mathbf{x}) = \mathbf{W}l(\mathbf{x})$$

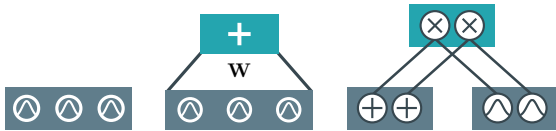


probabilistic circuits (PCs)

a tensorized definition

- I. A set of tractable functions is a circuit layer
- II. A linear projection of a layer is a circuit layer
- III. The product of two layers is a circuit layer

$$c(\mathbf{x}) = \mathbf{l}(\mathbf{x}) \odot \mathbf{r}(\mathbf{x}) \quad // \text{Hadamard}$$



probabilistic circuits (PCs)

a tensorized definition

- I. A set of tractable functions is a circuit layer
- II. A linear projection of a layer is a circuit layer
- III. The product of two layers is a circuit layer

$$c(\mathbf{x}) = \mathbf{l}(\mathbf{x}) \odot \mathbf{r}(\mathbf{x}) \quad // \text{Hadamard}$$

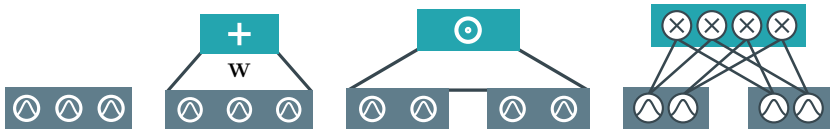


probabilistic circuits (PCs)

a tensorized definition

- I. A set of tractable functions is a circuit layer
- II. A linear projection of a layer is a circuit layer
- III. The product of two layers is a circuit layer

$$c(\mathbf{x}) = \text{vec}(\mathbf{l}(\mathbf{x})\mathbf{r}(\mathbf{x})^\top) \quad // \text{Kronecker}$$



probabilistic circuits (PCs)

a tensorized definition

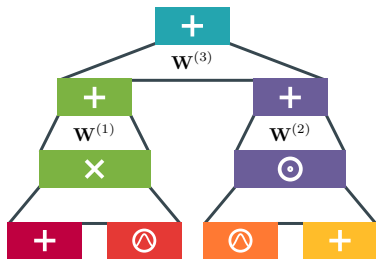
- I. A set of tractable functions is a circuit layer
- II. A linear projection of a layer is a circuit layer
- III. The product of two layers is a circuit layer

$$c(\mathbf{x}) = \text{vec}(\mathbf{l}(\mathbf{x})\mathbf{r}(\mathbf{x})^\top) \quad // \text{Kronecker}$$



probabilistic circuits (PCs)

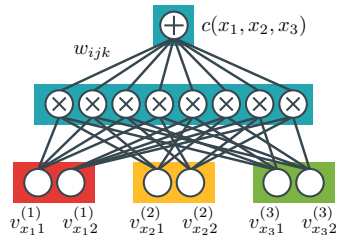
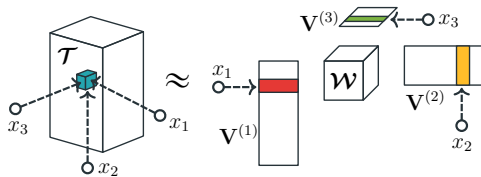
a tensorized definition



- I. A set of tractable functions is a circuit layer
 - II. A linear projection of a layer is a circuit layer
 - III. The product of two layers is a circuit layer
- stack layers to build a deep circuit!***

tensor factorizations

as circuits



Loconte et al., "What is the Relationship between Tensor Factorizations and Circuits (and How Can We Exploit it)?", 2025

wait...!

how do we **learn** them?

which parameters?

how to reparameterize circuits

Input distributions.

Sum unit parameters.

which parameters?

how to reparameterize circuits

Input distributions. Each input can be a different parametric distribution

⇒ *Bernoullis, Categoricals, Gaussians, **exponential families**, small NNs, ...*

Sum unit parameters.

which parameters?

how to reparameterize circuits

Input distributions. Each input can be a different parametric distribution

Sum unit parameters. Enforce them to be non-negative, i.e., $w_i \geq 0$ but unnormalized

$$w_i = \exp(\alpha_i), \quad \alpha_i \in \mathbb{R}, \quad i = 1, \dots, K$$

and renormalize the loss

$$\min_{\theta} - \left(\sum_{i=1}^N \log \tilde{p}_{\theta}(\mathbf{x}^{(i)}) - \log \int \tilde{p}_{\theta}(\mathbf{x}^{(i)}) d\mathbf{X} \right)$$

or just renormalize the weights, i.e., $\sum_i w_i = 1$

$$\mathbf{w} = \text{softmax}(\boldsymbol{\alpha}), \quad \boldsymbol{\alpha} \in \mathbb{R}^K$$

wait...!

how do we **learn** them?

wait...!

how do we **learn** them?

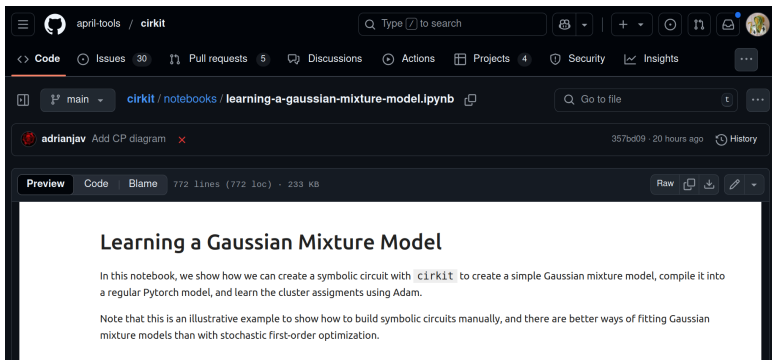
just SGD your way as usual!

⇒ or any other gradient-based optimizer



learning & reasoning with circuits in pytorch

`github.com/april-tools/circuit`

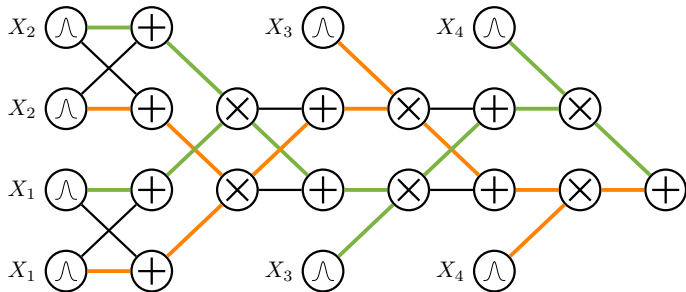


a notebook on learning GMMs as circuits

<https://github.com/april-tools/cirqit/blob/main/notebooks/learning-a-gaussian-mixture-model.ipynb>

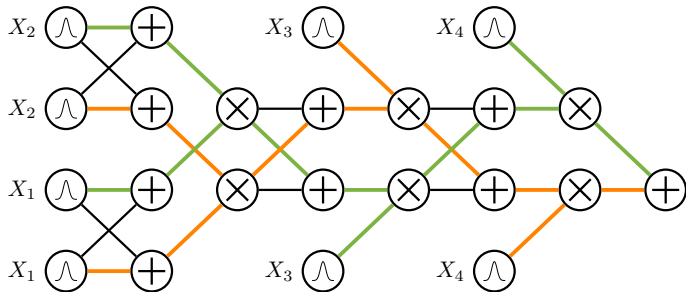
*what about **deep** circuits?*

deep mixtures



$$p(\mathbf{x}) = \sum_{\mathcal{T}} \left(\prod_{w_j \in \mathbf{w}_{\mathcal{T}}} w_j \right) \prod_{l \in \text{leaves}(\mathcal{T})} p_l(\mathbf{x})$$

deep mixtures



an exponential number of mixture components!

...why PCs?

1. A grammar for tractable models

One formalism to represent many probabilistic models

⇒ *#logic #Trees #XGBoost, Tensor Networks, ...*

...why PCs?

1. A grammar for tractable models

One formalism to represent many probabilistic models

⇒ #logic #Trees #XGBoost, Tensor Networks, ...

2. **Tractability** == **structural properties**!!!

Exact computations of reasoning tasks are certified by guaranteeing certain structural properties. #marginals #expectations #MAP, #product ...

again...?!

why tractability?

reasoning about ML models



q₁

“What is the probability of a treatment for a patient with **unavailable records**?”



q₂

“How **fair** is the prediction is a certain protected attribute changes?”



q₃

“Can we certify no **adversarial examples** exist?”

reasoning about ML models



q₁ $\int p(\mathbf{x}_o, \mathbf{x}_m) d\mathbf{X}_m$
(missing values)

q₂ $\mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=0)} [f_0(\mathbf{x}_c)] -$
 $\mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=1)} [f_1(\mathbf{x}_c)]$
(fairness)

q₃ $\mathbb{E}_{\mathbf{e} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_D)} [f(\mathbf{x} + \mathbf{e})]$
(adversarial robust.)

...in the language of probabilities

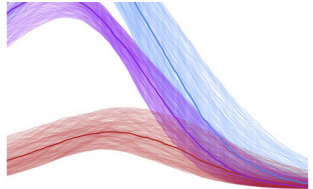
more complex reasoning



neuro-symbolic AI



probabilistic programming



computing uncertainties
(Bayesian inference)

...and more application scenarios

reasoning about ML models



q₁

$$\int p(\mathbf{x}_o, \mathbf{x}_m) d\mathbf{x}_m$$

(missing values)

q₂

$$\mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=0)} [f_0(\mathbf{x}_c)] - \mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=1)} [f_1(\mathbf{x}_c)]$$

(fairness)

q₃

$$\mathbb{E}_{\mathbf{e} \sim \mathcal{N}(0, \sigma^2 \mathbf{I}_D)} [f(\mathbf{x} + \mathbf{e})]$$

(adversarial robust.)

hard to compute in general!

reasoning about ML models



q₁ $\int p(\mathbf{x}_o, \mathbf{x}_m) d\mathbf{X}_m$
(missing values)

q₂ $\mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=0)} [f_0(\mathbf{x}_c)] -$
 $\mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=1)} [f_1(\mathbf{x}_c)]$
(fairness)

q₃ $\mathbb{E}_{\mathbf{e} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_D)} [f(\mathbf{x} + \mathbf{e})]$
(adversarial robust.)

*it is crucial we compute them **exactly** and in **polytime**!*

reasoning about ML models



q₁ $\int p(\mathbf{x}_o, \mathbf{x}_m) d\mathbf{X}_m$
(missing values)

q₂ $\mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=0)} [f_0(\mathbf{x}_c)] -$
 $\mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=1)} [f_1(\mathbf{x}_c)]$
(fairness)

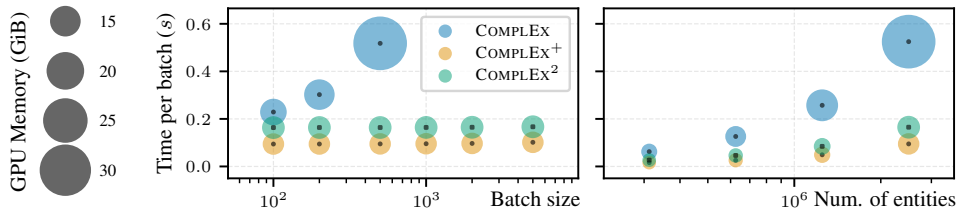
q₃ $\mathbb{E}_{\mathbf{e} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_D)} [f(\mathbf{x} + \mathbf{e})]$
(adversarial robust.)

*it is crucial we compute them **tractably!***

why tractable models?

they can be much **faster** than intractable ones!

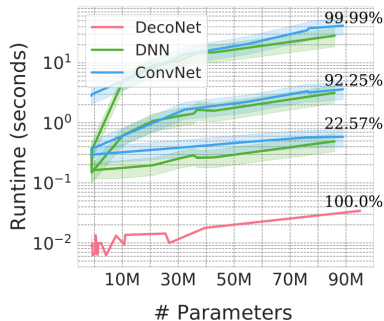
useful to scale to graph datasets with millions of entities



why tractable models?

they can be much **faster** than intractable ones!

one-shot computation of an **exact expectation** versus Monte Carlo estimates



q₃

$$\mathbb{E}_{\mathbf{e} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_D)} [f(\mathbf{x} + \mathbf{e})]$$

(adversarial robust.)

why?

tractable computations?

why?

*we always perform
tractable computations
in the end,
even after approximate inference!*



$$\min_{q \in \mathcal{Q}} \text{KL}(q||p)$$

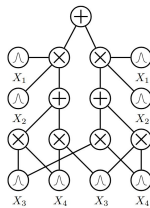
we have to pick a **tractable** variational distribution q ...otherwise we cannot compute its density, mode, marginals later

$\Rightarrow$ e.g., Gaussian, GMM, HMM, normalizing flow, etc

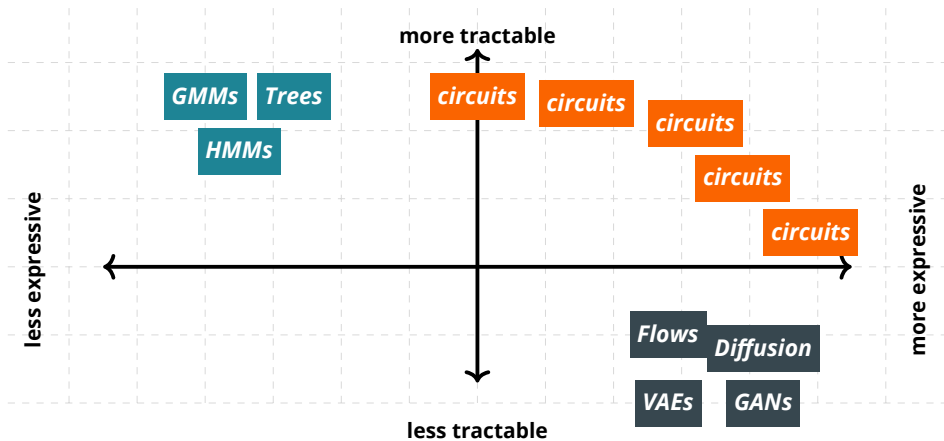
$\Rightarrow$ then, what is the largest tractable class we can use???

Goal

*“Can we find
a **middle ground**
between
tractability and expressiveness?”*



**(deep)
circuits**



***navigate the trade-off
between tractability and expressiveness***

structural properties

smoothness

decomposability

compatibility

determinism

the combination of certain structural properties

guarantees

tractable computation of certain query classes

structural properties

property A

property B

property C

property D

***the combination of certain
structural properties
guarantees
tractable computation of
certain query classes***

structural properties

property A

property B

property C

property D

tractable computation of *arbitrary integrals*

$$p(\mathbf{y}) = \int p(\mathbf{z}, \mathbf{y}) d\mathbf{Z}, \quad \forall \mathbf{Y} \subseteq \mathbf{X}, \quad \mathbf{Z} = \mathbf{X} \setminus \mathbf{Y}$$

$\Rightarrow$ *sufficient* and *necessary* conditions
for a single feedforward evaluation

$\Rightarrow$ tractable partition function

$\Rightarrow$ also any *conditional* is tractable

structural properties

smoothness

tractable computation of **arbitrary integrals**

decomposability

$$p(\mathbf{y}) = \int p(\mathbf{z}, \mathbf{y}) d\mathbf{Z}, \quad \forall \mathbf{Y} \subseteq \mathbf{X}, \quad \mathbf{Z} = \mathbf{X} \setminus \mathbf{Y}$$

property C

$\Rightarrow$ **sufficient** and **necessary** conditions
for a single feedforward evaluation

property D

$\Rightarrow$ tractable partition function
 $\Rightarrow$ also any **conditional** is tractable

structural properties

smoothness

smoothness $\wedge$ **decomposability** $\implies$ **multilinearity**

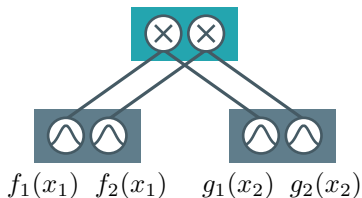
decomposability

property C

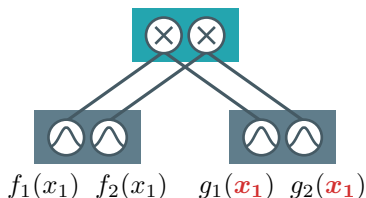
property D

multilinearity

the inputs of product units are defined over disjoint sets of variables



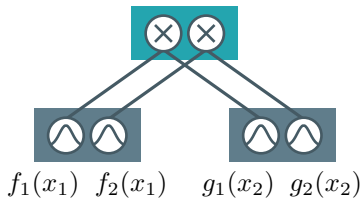
✓ **multilinear**



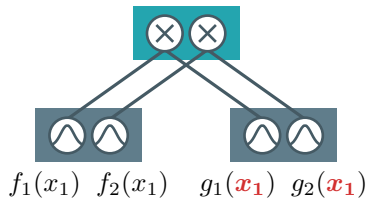
✗ **not multilinear**

multilinearity

the inputs of product units are defined over disjoint sets of variables



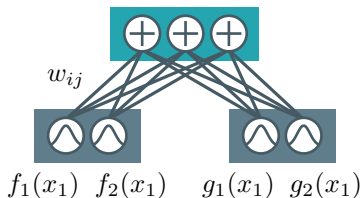
decomposable circuit



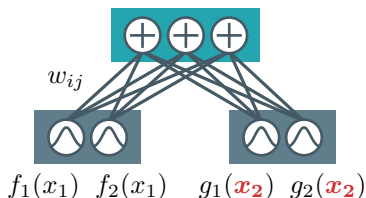
non-decomposable circuit

multilinearity

the inputs of sum units are defined over the same variables



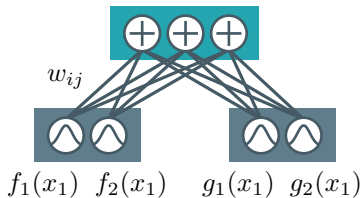
✓ **multilinear**



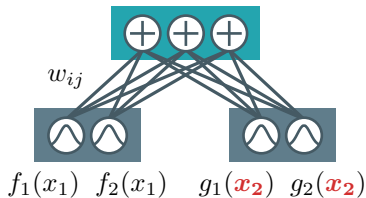
✗ **not multilinear**

multilinearity

the inputs of sum units are defined over the same variables



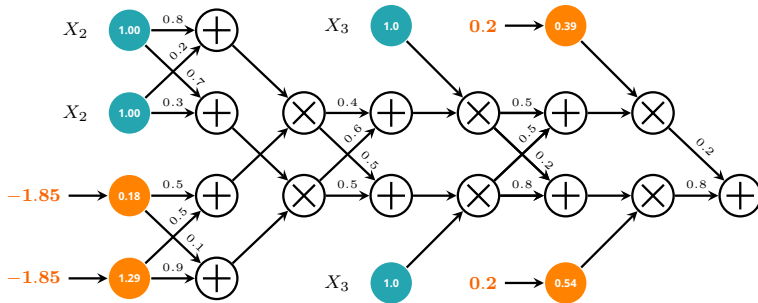
smooth circuit



non-smooth circuit

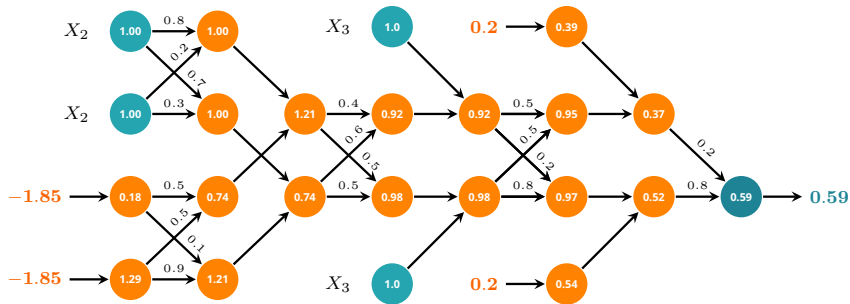
marginal queries = **feedforward** **evaluation**

$$p(X_1 = -1.85, X_4 = 0.2)$$



marginal queries = ***feedforward*** evaluation

$$p(X_1 = -1.85, X_4 = 0.2)$$

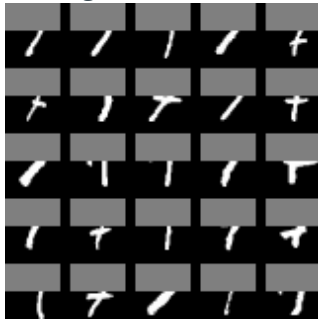


tractable marginals on PCs

Original



Missing



Conditional sample

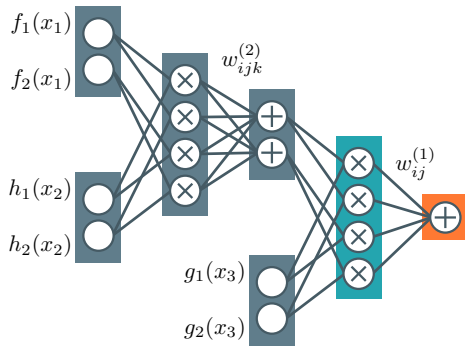


smooth + **decomposable** circuits = ...

compute arbitrary summations (or integrals)
 $\implies$ linear time in circuit size!

E.g., partition function $\sum_{\mathbf{x}} c(\mathbf{x})$

or $\int c(\mathbf{x}) \, d\mathbf{x}$ in the continuous case



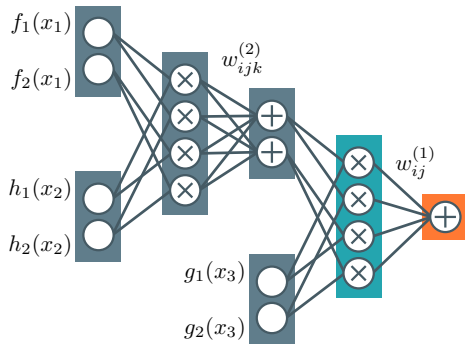
smooth + **decomposable** circuits = ...

$$\text{If } c(\mathbf{x}) = \sum_i w_i c_i(\mathbf{x})$$

(smoothness):

$$\begin{aligned} \int c(\mathbf{x}) \, d\mathbf{x} &= \int \sum_i w_i c_i(\mathbf{x}) \, d\mathbf{x} \\ &= \sum_i w_i \int c_i(\mathbf{x}) \, d\mathbf{x} \end{aligned}$$

⇒ integrals are “pushed down” to the inputs



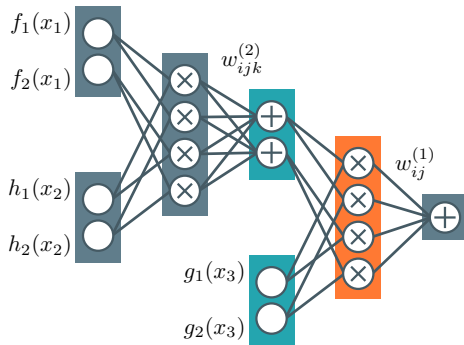
smooth + **decomposable** circuits = ...

$$\text{If } c(\mathbf{x}) = c_1(\mathbf{y}) c_2(\mathbf{z})$$

(decomposability):

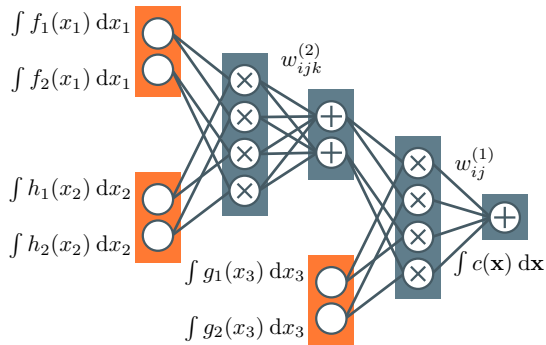
$$\begin{aligned} \int c(\mathbf{x}) d\mathbf{x} &= \int \int c_1(\mathbf{y}) c_2(\mathbf{z}) dy dz \\ &= \left(\int c_1(\mathbf{y}) dy \right) \left(\int c_2(\mathbf{z}) dz \right) \end{aligned}$$

$\Rightarrow$ integrals “decompose” into easier ones



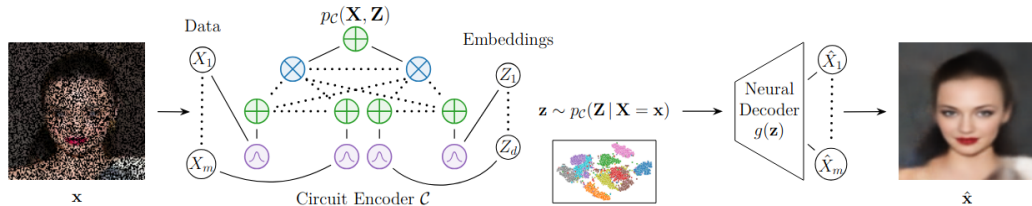
smooth + **decomposable** circuits = ...

Integrate simple input functions $f(x)$
 $\Rightarrow$ Gaussians, polynomials, splines, ...

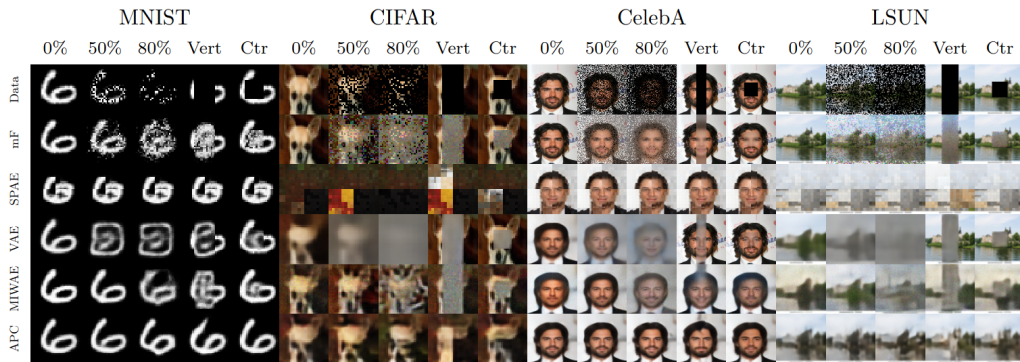


use tractable models
inside intractable pipelines
where it matters!

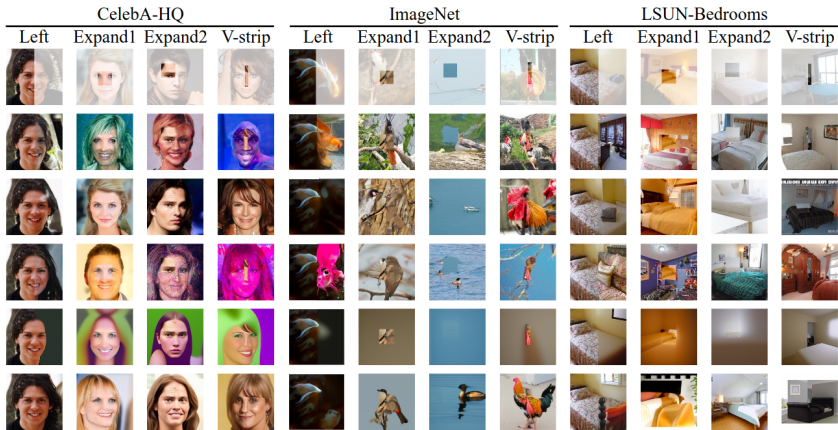
tractable + intractable



tractable conditioning over every missing mask



better than (V)AEs for missing values



which structural properties

for complex reasoning



q₁

$$\int p(\mathbf{x}_o, \mathbf{x}_m) d\mathbf{x}_m$$

(missing values)

*properties **A+B***



q₂

$$\mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=0)} [f_0(\mathbf{x}_c)] - \mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=1)} [f_1(\mathbf{x}_c)]$$

(fairness)

properties ???



q₃

$$\mathbb{E}_{\mathbf{e} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_D)} [f(\mathbf{x} + \mathbf{e})]$$

(adversarial robust.)

properties ???

structural properties

smoothness

decomposability

compatibility

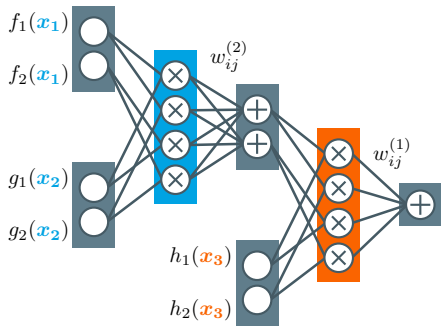
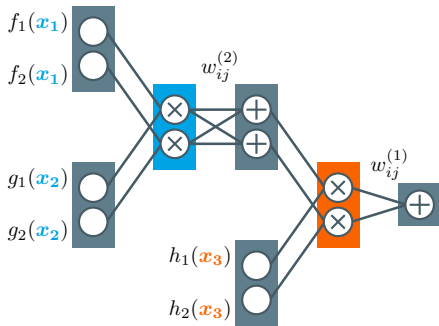
property D

Integrals involving two or more functions:
e.g., expectations

$$\mathbb{E}_{\mathbf{x} \sim p} f(\mathbf{x}) = \int p(\mathbf{x}) f(\mathbf{x}) d\mathbf{x}$$

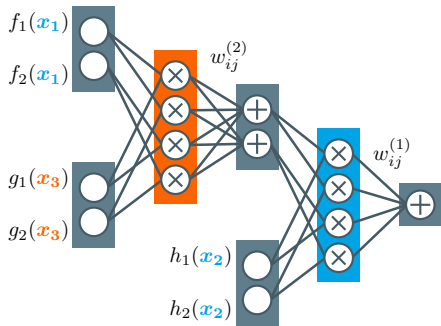
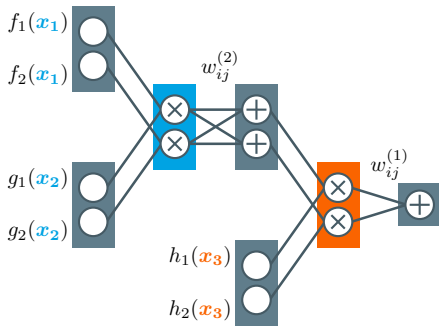
when both $p(\mathbf{x})$ and $f(\mathbf{x})$ are circuits

compatibility



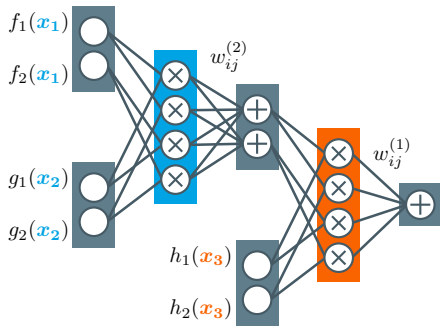
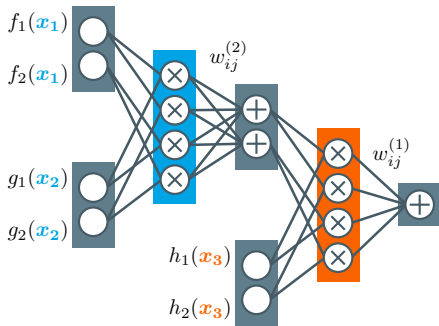
compatible circuits

compatibility



non-compatible circuits

compatibility



structured-composable = compatible with itself

structural properties

smoothness

decomposability

compatibility

property D

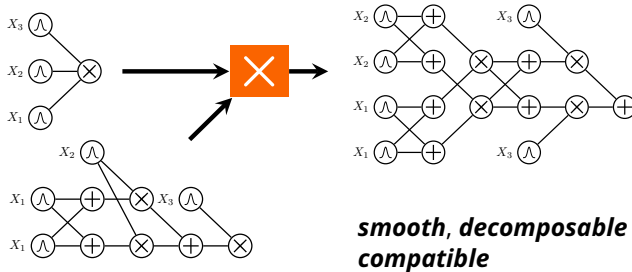
compatibility



smoothness $\wedge$ **decomposability**

compatibility $\Rightarrow$ tractable expectations

tractable products



$$\text{compute } \mathbb{E}_{\mathbf{x} \sim p} f(\mathbf{x}) = \int p(\mathbf{x}) f(\mathbf{x}) d\mathbf{x} \text{ in } O(|p| |f|)$$

which structural properties

for complex reasoning



q₁

$$\int p(\mathbf{x}_o, \mathbf{x}_m) d\mathbf{x}_m$$

(missing values)

*properties **A+B***



q₂

$$\mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=0)} [f_0(\mathbf{x}_c)] - \mathbb{E}_{\mathbf{x}_c \sim p(\mathbf{x}_c | X_s=1)} [f_1(\mathbf{x}_c)]$$

(fairness)

*property **C***



q₃

$$\mathbb{E}_{\mathbf{e} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_D)} [f(\mathbf{x} + \mathbf{e})]$$

(adversarial robust.)

*property **B***



```
1 from cirq.symbolic.circuit import Circuit
2 from cirq.symbolic.functional import (
3     integrate, multiply)
4
5 # Circuits expectation \int [p(x) f(x)]dx
6 def expectation(p: Circuit, f: Circuit) -> Circuit:
7     i = multiply(p, f)
8     return integrate(i)
9
10 # Squared loss \int [p(x)-q(x)]^2dx = E_p[p] + E_q[q] - 2E_p[q]
11 #           = \int p^2(x)dx + \int q^2(x)dx - 2\int p(x)q(x)dx
12 def squared_loss(p: Circuit, q: Circuit) -> Circuit:
13     p2 = multiply(p, p)
14     q2 = multiply(q, q)
15     pq = multiply(p, q)
16     return integrate(p2) + integrate(q2) - 2 * integrate(pq)
```

structural properties

smoothness

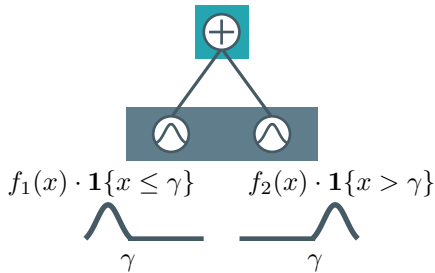
decomposability

compatibility

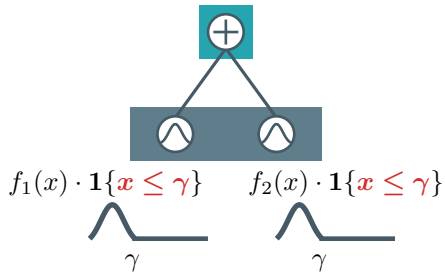
determinism

determinism

the inputs of sum units are defined over disjoint supports



deterministic circuit

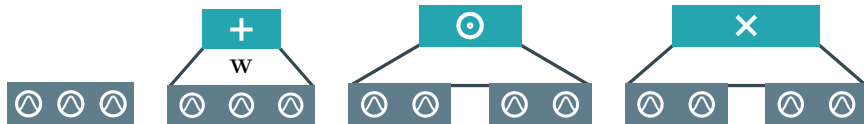


non-deterministic circuit

*how to build **deep** circuits?*

probabilistic circuits (PCs)

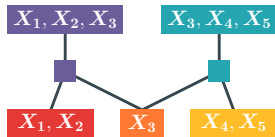
the layer-wise definition



```
1 from cirkit.templates import circuit_templates
2
3 symbolic_circuit = circuit_templates.image_data(
4     (1, 28, 28),          # The shape of MNIST
5     region_graph='quad-graph',
6     input_layer='categorical', # input distributions
7     sum_product_layer='cp',    # CP, Tucker, CP-T
8     num_input_units=64,       # overparameterizing
9     num_sum_units=64,
10    sum_weight_param=circuit_templates.Parameterization(
11        activation='softmax',
12        initialization='normal'
13    )
14 )
```

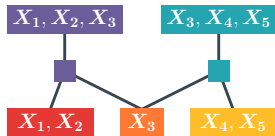
```
1 from cirkit.templates import circuit_templates
2
3 symbolic_circuit = circuit_templates.image_data(
4     (1, 28, 28),          # The shape of MNIST
5     region_graph='quad-graph',
6     input_layer='categorical', # input distributions
7     sum_product_layer='cp',    # CP, Tucker, CP-T
8     num_input_units=64,       # overparameterizing
9     num_sum_units=64,
10    sum_weight_param=circuit_templates.Parameterization(
11        activation='softmax',
12        initialization='normal'
13    )
14 )
```

learning recipe

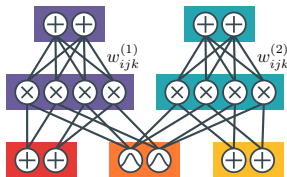


1) Build a *region graph*

learning recipe



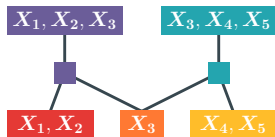
1) Build a *region graph*



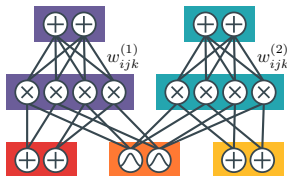
2) Overparameterize

- 2.1)** pick a (composite) layer type
- 2.2)** choose how many units per layer

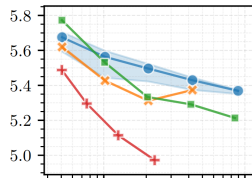
learning recipe



1) Build a *region graph*



2) Overparameterize

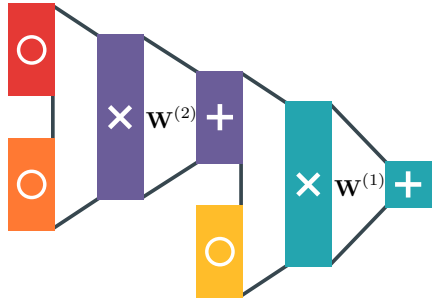
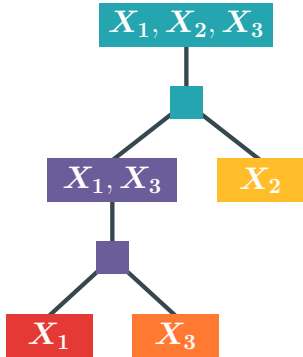


3) Learn parameters

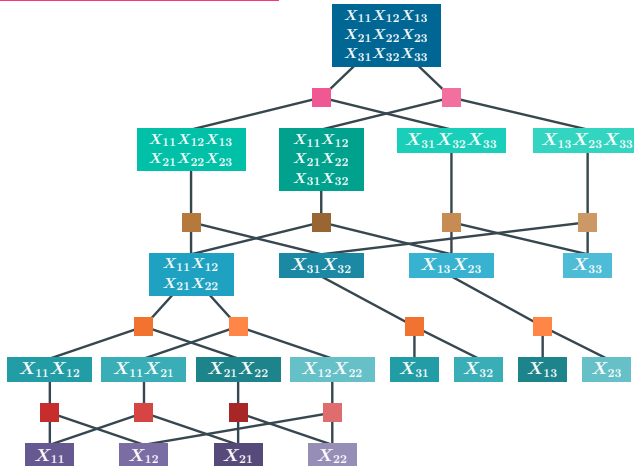
use any optimizer in pytorch

region graphs

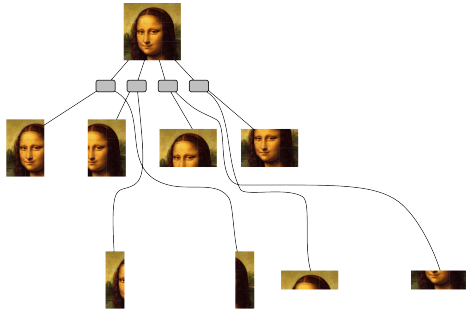
a template for smooth&decomposable PCs

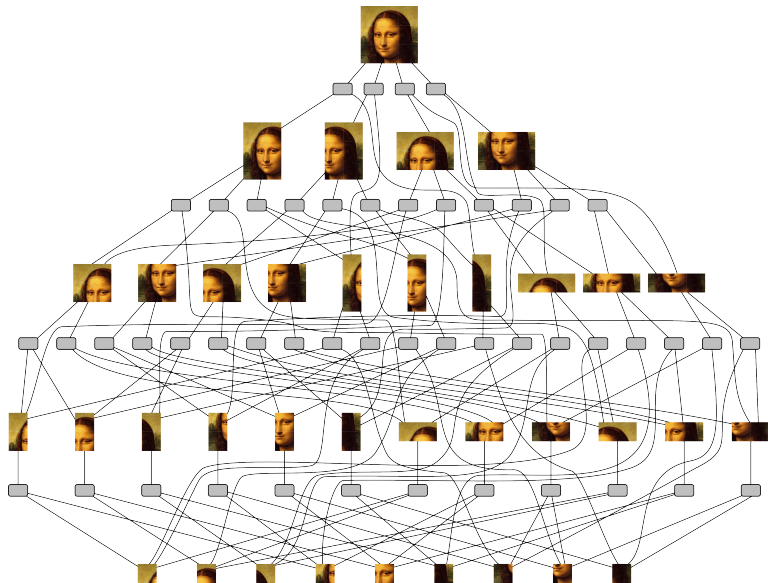


which region graph?





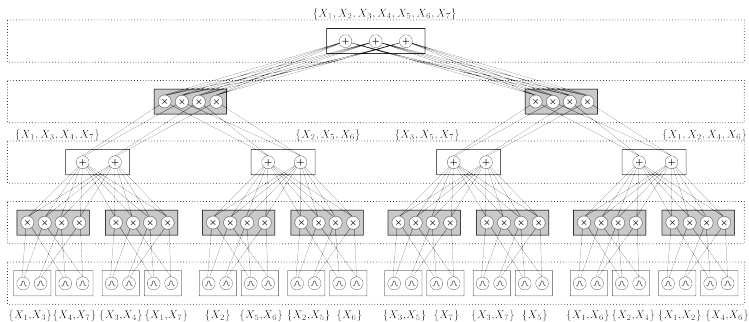




random regions graphs

The “no-learning” option

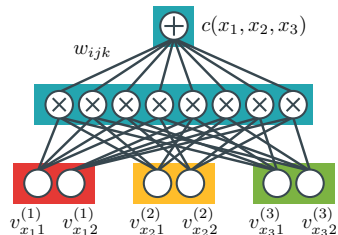
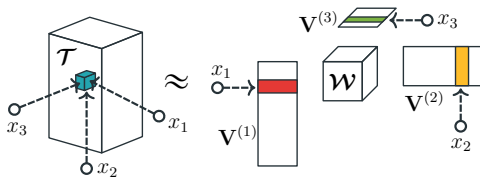
Generating a random region graph, by recursively splitting $\mathbf{X}$ into two random parts:



```
1 from cirkit.templates import circuit_templates
2
3 symbolic_circuit = circuit_templates.image_data(
4     (1, 28, 28),          # The shape of MNIST
5     region_graph='quad-graph',
6     input_layer='categorical', # input distributions
7     sum_product_layer='cp',    # CP, Tucker, CP-T
8     num_input_units=64,        # overparameterizing
9     num_sum_units=64,
10    sum_weight_param=circuit_templates.Parameterization(
11        activation='softmax',
12        initialization='normal'
13    )
14 )
```

circuits layers

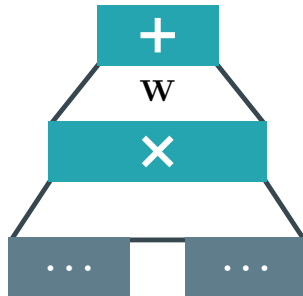
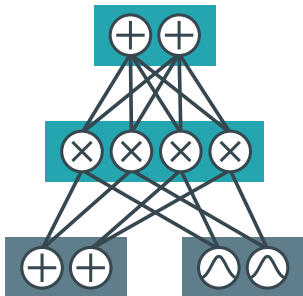
as tensor factorizations



Loconte et al., "What is the Relationship between Tensor Factorizations and Circuits (and How Can We Exploit it)?", 2025

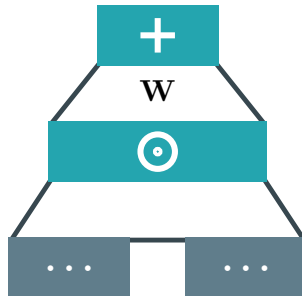
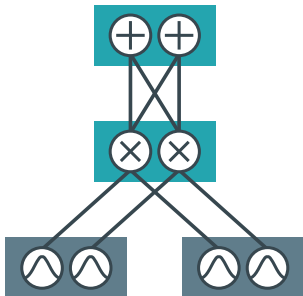
more layers

Tucker decomposition

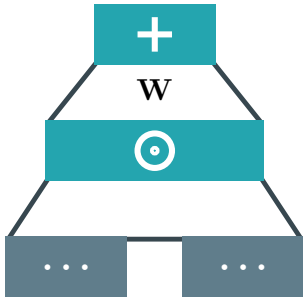


more layers

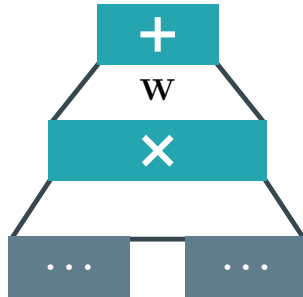
Candecomp Parafac (CP) decomposition



more layers



CP layer



Tucker layer



a notebook on learning a deep circuit on MNIST

<https://github.com/april-tools/cirkit/blob/main/notebooks/learning-a-circuit.ipynb>



loreloc updated notebooks with respect to API changes ✖ e3e7e80 · 2 days ago

Preview Code Blame 1082 lines (1082 loc) · 793 KB Raw  

Notebook on Region Graphs and Sum Product Layers

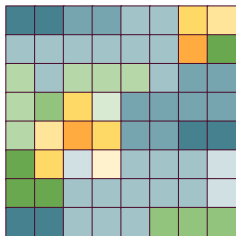
Goals

By the end of this tutorial you will:

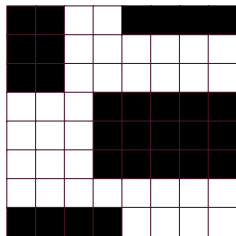
- [know what a region graph is](#)
- know how to [choose between region graphs](#) for your circuit
- understand how to parametrize a circuit by [choosing a sum product layer](#)
- build circuits to **tractably** estimate a [probability distribution over images](#)¹

mix& match your RGs and layers

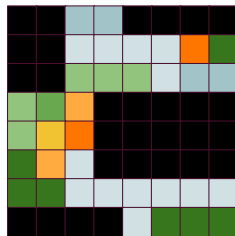
`https://github.com/april-tools/cirkit/blob/main/notebooks/region-graphs-and-parametrisation.ipynb`



$q(\mathbf{x})$

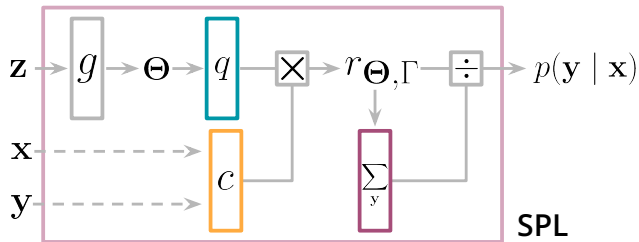


$c(\mathbf{x})$



$$q(\mathbf{x}) \cdot c(\mathbf{x}) \quad \frac{q(\mathbf{x}) \cdot c(\mathbf{x})}{\sum_{\mathbf{x}} q(\mathbf{x}) \cdot c(\mathbf{x})}$$

we have all the ingredients we need



$$p(\mathbf{y} \mid \mathbf{x}) = \mathbf{q}_{\Theta}(\mathbf{y} \mid g(\mathbf{z})) \cdot \mathbf{c}_{\mathbf{K}}(\mathbf{x}, \mathbf{y}) / \mathbf{Z}(\mathbf{x})$$

$$\mathbf{Z}(\mathbf{x}) = \sum_{\mathbf{y}} \mathbf{q}_{\Theta}(\mathbf{y} \mid \mathbf{x}) \cdot \mathbf{c}_{\mathbf{K}}(\mathbf{x}, \mathbf{y})$$

SPL



Ground Truth



ResNet-18



Semantic Loss



circuits

predictions guarantee a logical constraint 100% of the time!

SPL recipe

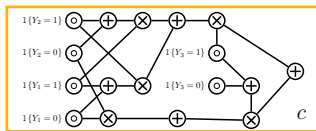
$$K : (Y_1 = 1 \implies Y_3 = 1)$$

$$\wedge (Y_2 = 1 \implies Y_3 = 1)$$

1) Take a
logical constraint

SPL recipe

$$K : (Y_1 = 1 \implies Y_3 = 1) \\ \wedge (Y_2 = 1 \implies Y_3 = 1)$$

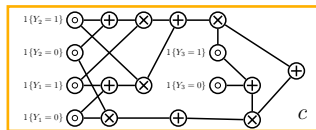


1) Take a
logical constraint

2) Compile it into
a constraint circuit

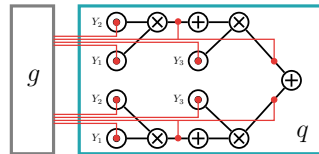
SPL recipe

$$K : (Y_1 = 1 \implies Y_3 = 1) \\ \wedge (Y_2 = 1 \implies Y_3 = 1)$$



1) Take a
logical constraint

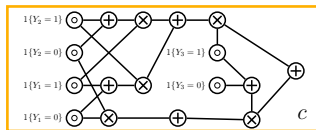
2) Compile it into
a constraint circuit



3) Multiply it
by a circuit distribution

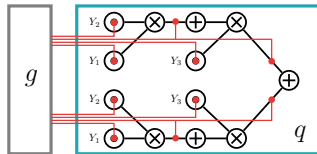
SPL recipe

$$\begin{aligned} K : & (Y_1 = 1 \implies Y_3 = 1) \\ \wedge & (Y_2 = 1 \implies Y_3 = 1) \end{aligned}$$



1) Take a
logical constraint

2) Compile it into
a constraint circuit



3) Multiply it
by a circuit distribution

4) train end-to-end by sgd!

Experiments



Architecture	Exact	Hamming	Consistent
ResNet-18+FIL	55.0	97.7	56.9
ResNet-18+ $\mathcal{L}_{SL}$	59.4	97.7	61.2
ResNet-18+SPL	78.2	96.3	100.0

Experiments

Ground Truth



cost: 39.31



cost: 57.31

FIL



cost: ∞



cost: ∞

$\mathcal{L}_{SL}$

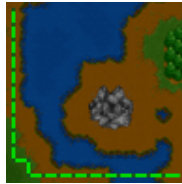


cost: ∞

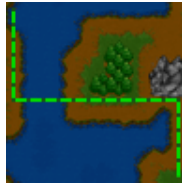


cost: ∞

SPL



cost: 45.09



cost: 58.09

SPL

(and variants)

everywhere

DeepProbLog: Neural Probabilistic Logic Programming

Robin Manhaeve
KU Leuven
`robin.manhaeve@cs.kuleuven.be`

Sebastijan Dumančić
KU Leuven
`sebastijan.dumancic@cs.kuleuven.be`

Angelika Kimmig
Cardiff University
`KimmigA@cardiff.ac.uk`

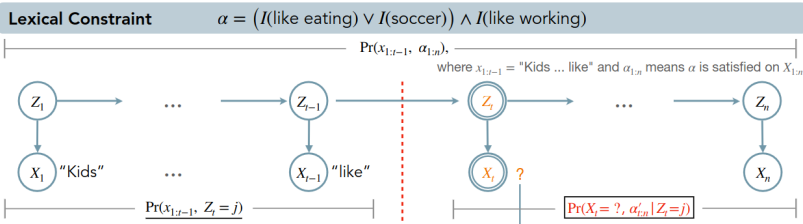
Thomas Demeester*
Ghent University - imec
`thomas.demeester@ugent.be`

Luc De Raedt*
KU Leuven
`luc.deraedt@cs.kuleuven.be`

the “father” of NeSy (NeurIPS 2018)

Tractable Control for Autoregressive Language Generation

Honghua Zhang^{*1} Meihua Dang^{*1} Nanyun Peng¹ Guy Van den Broeck¹



constrained text generation with LLMs (ICML 2023)

Adaptable Logical Control for Large Language Models

Honghua Zhang*
UCLA
hzhang19@cs.ucla.edu

Po-Nien Kung*
UCLA
ponienkung@cs.ucla.edu

Masahiro Yoshida†
UCLA & Sony Group Corporation
masahiroyoshida@ucla.edu

Guy Van den Broeck
UCLA
guyvdb@cs.ucla.edu

Nanyun Peng
UCLA
violetpeng@cs.ucla.edu

User: given the following context, generate infilling text for [BLANK] using key phrases "alien mothership", "far from over"; generated text must contain 25 - 30 words.

"First they've defeated a small squad [BLANK] are few humans left, and despite their magical power, their numbers are getting fewer."

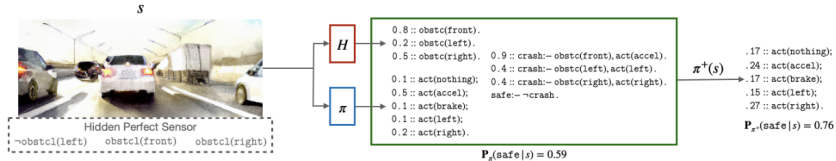
```
from CtrlG import *  
  
prefix = "First they defeated a ..."  
suffix = "are few humans left ..."  
  
dfa_list = [  
    DFA_all_of("alien mothership",  
              "far from over"),  
    DFA_word_count(25, 30),  
]  
dfa = DFA_logical_and(dfa_list)  
  
lp = CtrlGLogitsProcessor(  
    dfa, hmm, prefix, suffix)  
llm.generate(logits_processor=lp)
```

5 lines of code!

"First they've defeated a small squad of aliens, then a larger fleet of their ships. Eventually they've even managed to take down the alien mothership. But their problems are far from over. There are few humans left, and despite their magical power, their numbers are getting fewer."

Safe Reinforcement Learning via Probabilistic Logic Shields

Wen-Chi Yang¹, Giuseppe Marra¹, Gavin Rens and Luc De Raedt^{1,2}



reliable reinforcement learning (AAAI 23)

Logically Consistent Language Models via Neuro-Symbolic Integration

 = LLaMa 2  = LLaMa 2 	Forward Implication	Reverse Implication	Negation
	$A \rightarrow B$ A: (albatross, isA, bird) B: (albatross, isA, fish)	$\neg B \rightarrow \neg A$ B: (albatross, isNotA, organism) A: (albatross, isNotA, living thing)	$A \leftrightarrow A$ A: (computer, isA, airplane) A: (computer, isNotA, airplane)
	Is an albatross a bird?   Yes. Is an albatross a fish?   Yes. Logical:  Factual:   No. Logical:  Factual: 	Is it true that an albatross is not an organism?   No. Is it true that an albatross is not a living thing?   Yes. Logical:  Factual:   No. Logical:  Factual: 	Is a computer a airplane?   No. Is it true that a computer is not a airplane?   No. Logical:  Factual:   Yes. Logical:  Factual: 

improving logical (self-)consistency in LLMs at ICLR 2025

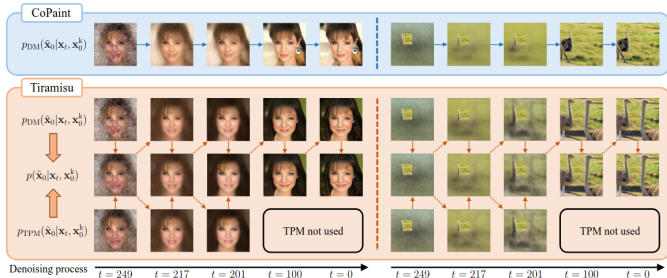
IMAGE INPAINTING VIA TRACTABLE STEERING OF DIFFUSION MODELS

Anji Liu¹, Mathias Niepert², Guy Van den Broeck¹

¹Department of Computer Science, University of California, Los Angeles

²Department of Computer Science, University of Stuttgart

liuanji@cs.ucla.edu, mathias.niepert@simtech.uni-stuttgart.de,
guyvdb@cs.ucla.edu,



How to Turn Your Knowledge Graph Embeddings into Generative Models

Lorenzo Loconte
University of Edinburgh, UK
l.loconte@sms.ed.ac.uk

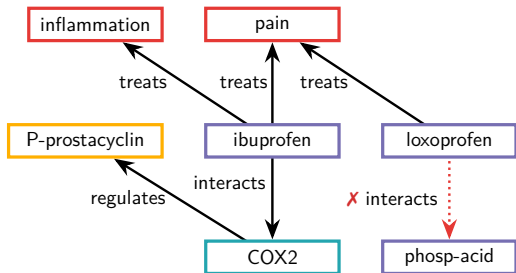
Nicola Di Mauro
University of Bari, Italy
nicola.dimauro@uniba.it

Robert Peharz
TU Graz, Austria
robert.peharz@tugraz.at

Antonio Vergari
University of Edinburgh, UK
avergari@ed.ac.uk

***enforce constraints in knowledge graph embeddings
oral at NeurIPS 2023***

constraints in KGs



- Drugs
- Symptoms
- Proteins
- Functions

K : only drugs and proteins interact

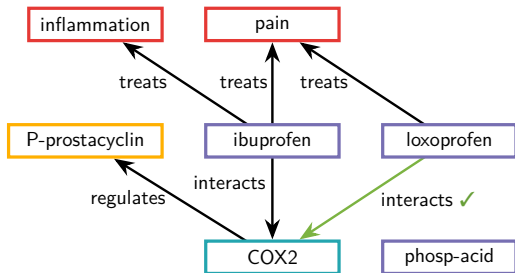
$\mathcal{A}$: $\langle \text{loxoprofen}, \text{interacts}, \text{phosp-acid} \rangle$



$\mathcal{A}$: $\langle \text{loxoprofen}, \text{interacts}, \text{COX2} \rangle$



constraints in KGs



- Drugs
- Symptoms
- Proteins
- Functions

K : only drugs and proteins interact

$\mathcal{A}$: $\langle \text{loxoprofen}, \text{interacts}, \text{phosp-acid} \rangle$

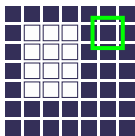
✗

$\mathcal{A}$: $\langle \text{loxoprofen}, \text{interacts}, \text{COX2} \rangle$

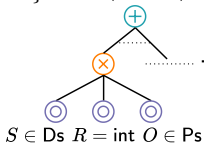
✓

guaranteed satisfaction of constraints

$$\mathbb{1}\{(S, \text{interacts}, O) \models K\}$$

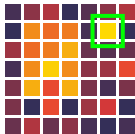


$$c_K(S, R, O)$$

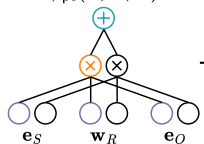


K : only drugs and proteins interact

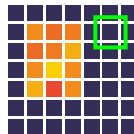
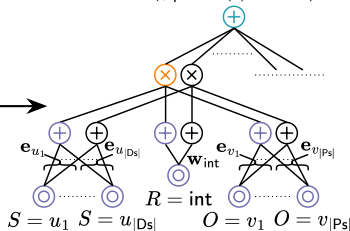
$$\phi_{\text{pc}}(S, \text{interacts}, O)$$



$$\phi_{\text{pc}}(S, R, O)$$



$$(\phi_{\text{pc}} \cdot c_K)(S, R, O)$$

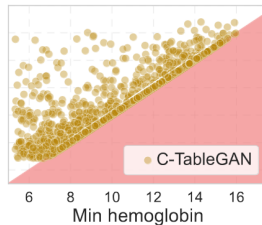
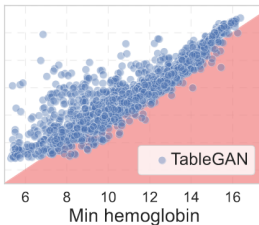
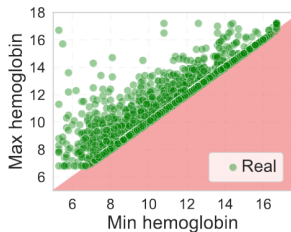


$$p_K(\text{loxoprofen, interacts, phosp-acid}) = 0$$

open problems

I constraints over continuous variables

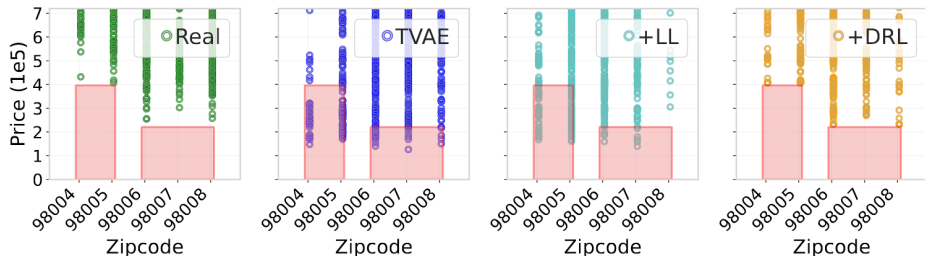
constraints over continuous variables



sampling within **convex** constraints

Stoian et al., "How Realistic Is Your Synthetic Data? Constraining Deep Generative Models for Tabular Data", [ICLR](#), 2024

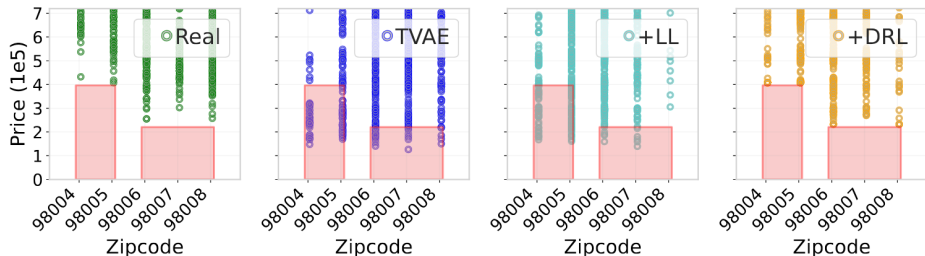
constraints over continuous variables



sampling within **non-convex** constraints

Stoian and Giunchiglia, "Beyond the convexity assumption: Realistic tabular data generation under quantifier-free real linear constraints", ICLR, 2025

constraints over continuous variables



sampling within **non-convex** constraints...**what about probabilities???**

Stoian and Giunchiglia, "Beyond the convexity assumption: Realistic tabular data generation under quantifier-free real linear constraints", ICLR, 2025

A Probabilistic Neuro-symbolic Layer for Algebraic Constraint Satisfaction

Leander Kurscheidt¹

Paolo Morettin²

Roberto Sebastiani²

Andrea Passerini²

Antonio Vergari¹

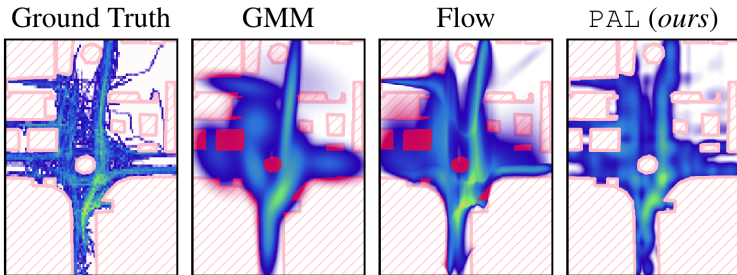
¹School of Informatics, University of Edinburgh, UK

²DiSI, University of Trento, Italy

compute exact constrained probabilities
best student runner-up paper at UAI 2025

constraints over continuous variables

much harder than the discrete case



guarantee the satisfaction of given non-convex constraints

Kurscheidt et al., "A Probabilistic Neuro-symbolic Layer for Algebraic Constraint Satisfaction", UAI, 2025

open problems

I constraints over continuous variables

II scaling to H U G E constraints

open problems

I constraints over continuous variables

II scaling to H U G E constraints

open problems

I constraints over continuous variables

II scaling to H U G E constraints

III learn (partial) constraints

IV revise constraints (continual learning)

wait...!

***“Given a reasoning task
can we **automatically** distill
a **tractable** algorithm for it?”***

A language for queries

Integral expressions that can be formed by composing these operators

$+$, $\times$, **pow**, **log**, **exp** and $/$

$\Rightarrow$ *many divergences and information-theoretic queries*

A language for queries

Integral expressions that can be formed by composing these operators

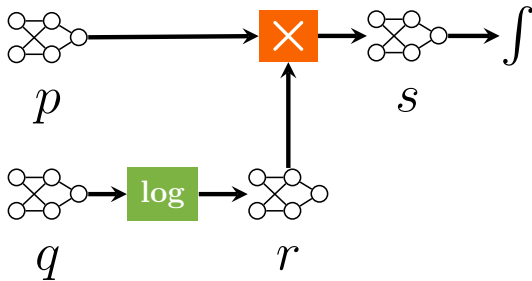
$+$, $\times$, `pow`, `log`, `exp` and $/$

$\Rightarrow$ many divergences and information-theoretic queries

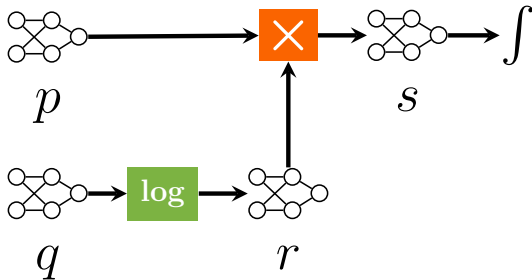
Represented as **higher-order computational graphs**—pipelines—operating over circuits!

$\Rightarrow$ re-using intermediate transformations across queries

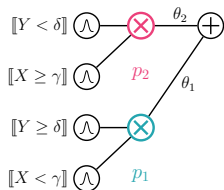
$$\text{XENT}(p \parallel q) = \int p(\mathbf{x}) \times \log q(\mathbf{x}) d\mathbf{X}$$



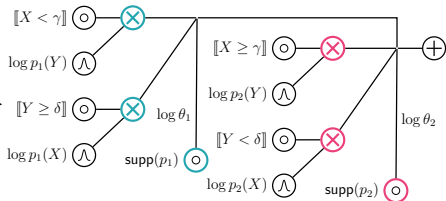
$$\text{XENT}(p \parallel q) = \int p(\mathbf{x}) \times \log q(\mathbf{x}) d\mathbf{X}$$



Tractable operators



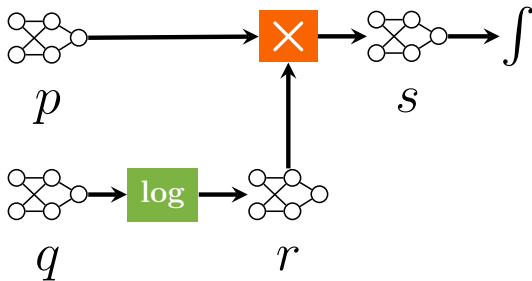
log



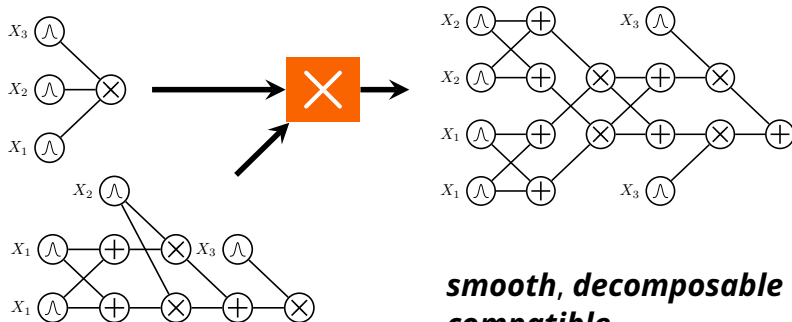
**smooth, decomposable
deterministic**

smooth, decomposable

$$\text{XENT}(p \parallel q) = \int p(\mathbf{x}) \times \log q(\mathbf{x}) d\mathbf{X}$$

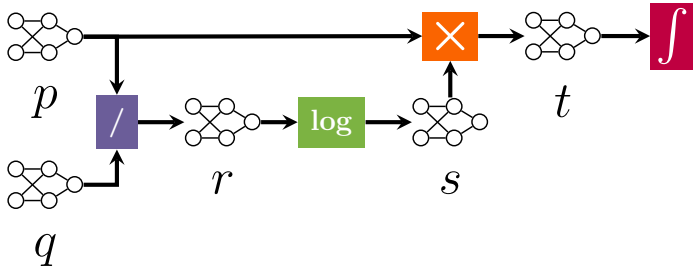


Tractable operators



***smooth, decomposable
compatible***

$$\int p(\mathbf{x}) \times \log \left(p(\mathbf{x}) / q(\mathbf{x}) \right) d\mathbf{X}$$



build a LEGO-like query calculus...

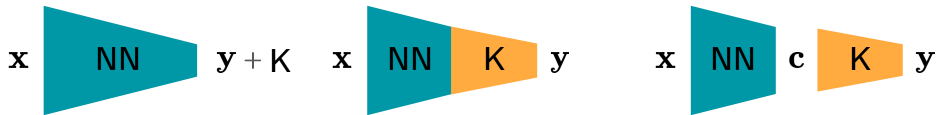
	Query	Tract. Conditions	Hardness
CROSS ENTROPY	$-\int p(\mathbf{x}) \log q(\mathbf{x}) d\mathbf{X}$	Cmp, q Det	#P-hard w/o Det
SHANNON ENTROPY	$-\sum p(\mathbf{x}) \log p(\mathbf{x})$	Sm, Dec, Det	coNP-hard w/o Det
RÉNYI ENTROPY	$(1 - \alpha)^{-1} \log \int p^\alpha(\mathbf{x}) d\mathbf{X}, \alpha \in \mathbb{N}$	SD	#P-hard w/o SD
MUTUAL INFORMATION	$(1 - \alpha)^{-1} \log \int p^\alpha(\mathbf{x}) d\mathbf{X}, \alpha \in \mathbb{R}_+$	Sm, Dec, Det	#P-hard w/o Det
KULLBACK-LEIBLER DIV.	$\int p(\mathbf{x}, \mathbf{y}) \log(p(\mathbf{x}, \mathbf{y}) / (p(\mathbf{x})p(\mathbf{y})))$	Sm, SD, Det*	coNP-hard w/o SD
RÉNYI'S ALPHA DIV.	$\int p(\mathbf{x}) \log(p(\mathbf{x}) / q(\mathbf{x})) d\mathbf{X}$	Cmp, Det	#P-hard w/o Det
ITAKURA-SAITO DIV.	$(1 - \alpha)^{-1} \log \int p^\alpha(\mathbf{x}) q^{1-\alpha}(\mathbf{x}) d\mathbf{X}, \alpha \in \mathbb{N}$	Cmp, q Det	#P-hard w/o Det
CAUCHY-SCHWARZ DIV.	$(1 - \alpha)^{-1} \log \int p^\alpha(\mathbf{x}) q^{1-\alpha}(\mathbf{x}) d\mathbf{X}, \alpha \in \mathbb{R}$	Cmp, Det	#P-hard w/o Det
SQUARED LOSS	$\int [p(\mathbf{x}) / q(\mathbf{x}) - \log(p(\mathbf{x}) / q(\mathbf{x})) - 1] d\mathbf{X}$	Cmp, Det	#P-hard w/o Det
	$-\log \frac{\int p(\mathbf{x}) q(\mathbf{x}) d\mathbf{X}}{\sqrt{\int p^2(\mathbf{x}) d\mathbf{X} \int q^2(\mathbf{x}) d\mathbf{X}}}$	Cmp	#P-hard w/o Cmp
	$\int (p(\mathbf{x}) - q(\mathbf{x}))^2 d\mathbf{X}$	Cmp	#P-hard w/o Cmp

*...and **compositionally** derive many more tractable algorithms*



learning & reasoning with circuits in pytorch

`github.com/april-tools/circuit`



questions?